



TaskFlow

Ứng dụng quản lý công việc cá nhân trên Android

Dữ liệu nằm trên máy, chạy offline, khoá bằng PIN hoặc vân tay, sao lưu bằng file JSON. Viết bằng Kotlin theo kiến trúc MVVM.

Nhóm thực hiện

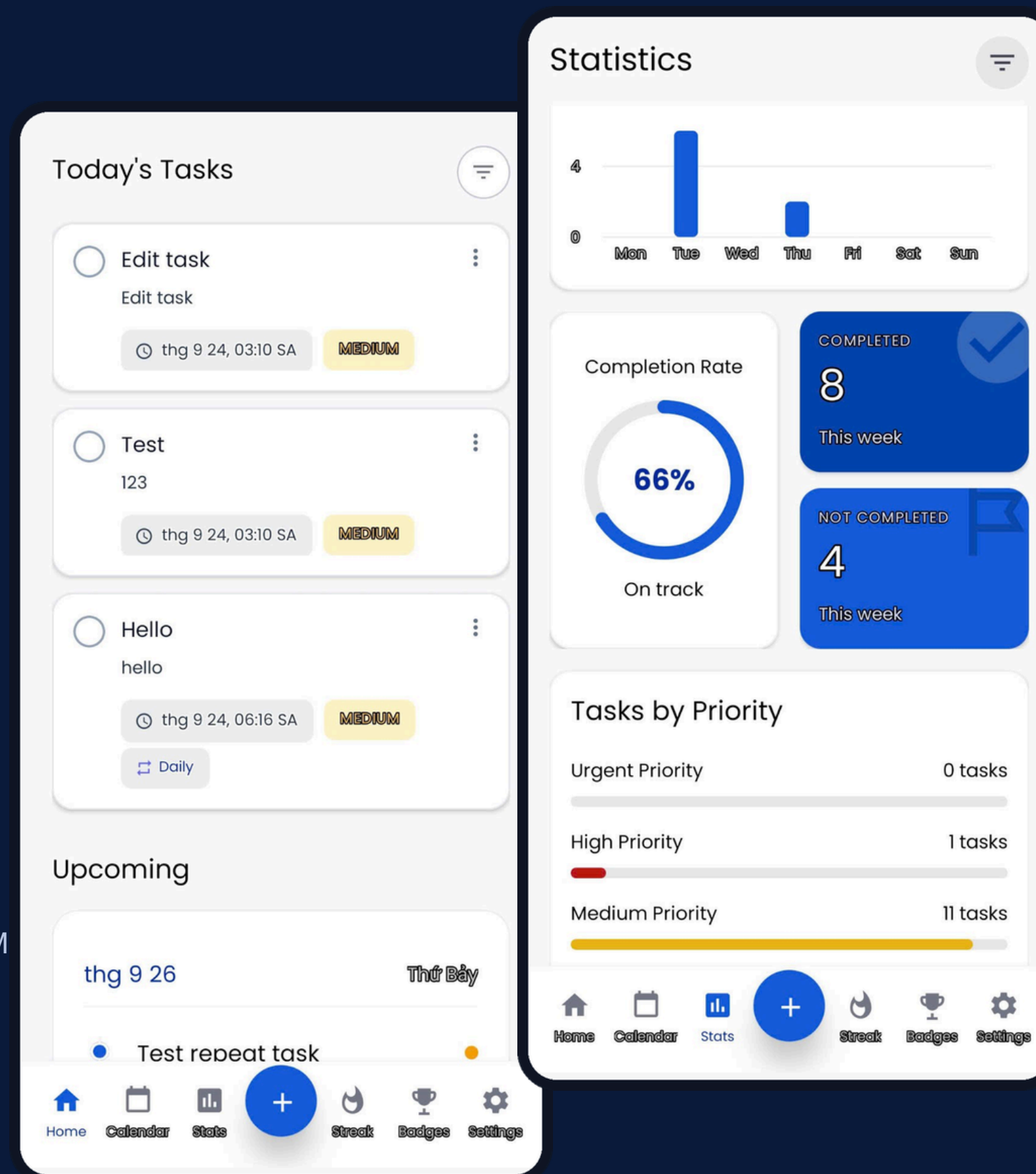
Trần Văn Thái
Trần Văn Ngọc Thắng
Nguyễn Lê Huy Tâm
Huỳnh Đình Chấn
Nguyễn Ngọc Gia Bảo

Giảng viên hướng dẫn

ThS. Mai Ngọc Châu

Học phần

Lập trình thiết bị di động
Khoa CNTT · ĐH Giao thông Vận tải TP.HCM



Nội dung và người trình bày

Mỗi thành viên trình bày đúng phần tính năng mình trực tiếp viết code.

PHẦN	NGƯỜI TRÌNH BÀY	NỘI DUNG	SLIDE
1	● Trần Văn Thái	Tổng quan, technical design, kiến trúc MVVM, cơ sở dữ liệu, khoá PIN, widget	3 – 12
2	● Trần Văn Ngọc Thắng	Tạo / sửa task, lịch tháng, mở khoá vân tay, App Shortcuts	13 – 17
3	● Nguyễn Lê Huy Tâm	Task lặp lại, import JSON, streak và 7 huy hiệu	18 – 23
4	● Huỳnh Đình Chấn	Pomodoro chạy nền, xoá / quá hạn / đổi giờ, backup & restore	24 – 28
5	● Nguyễn Ngọc Gia Bảo	Nhắc nhở đúng giờ, bộ lọc, trợ lý AI, kiểm thử	29 – 34
	● Trần Văn Thái	Nguyên tắc thiết kế & nợ kỹ thuật, tổng kết, demo và hỏi đáp	35 – 37

TaskFlow làm được gì

11 yêu cầu bắt buộc của đề tài và 6 phân hệ mở rộng. Mọi thứ chạy offline, trừ trợ lý AI.

Yêu cầu bắt buộc (11)

Tạo, xem, sửa, xoá task	Thắng · Chấn
Hạn chót, độ ưu tiên, trạng thái	Thắng
Lọc và sắp xếp	Bảo · Chấn
Nhắc nhở bằng AlarmManager	Bảo
Lưu trữ bằng Room	Thái
Khoá ứng dụng bằng PIN	Thái
Kiểm tra dữ liệu nhập, xử lý quá hạn	Bảo · Chấn
Task lặp lại	Tâm
Xem theo lịch	Thắng
Khôi phục nhắc sau reboot / đổi giờ	Thái · Chấn
Backup / restore JSON qua SAF	Thái · Chấn · Tâm

Phần mở rộng (6)

Widget màn hình chính tick hoàn thành không cần mở app	Thái
Thống kê Bento tỷ lệ hoàn thành, năng suất 7 ngày	Thái
Pomodoro Timer Foreground Service, lưu phiên tập trung	Chấn
Streak & huy hiệu chuỗi ngày, 7 huy hiệu đặc biệt	Tâm
Trợ lý AI ra lệnh bằng câu tự nhiên (Gemini)	Bảo
Vân tay & App Shortcuts mở khoá nhanh, vào thẳng chức năng	Thắng

Công nghệ và cách tổ chức mã nguồn

Một module Android, 100 % Kotlin, 108 file chia theo trách nhiệm. Không có backend: Room trên máy là nguồn dữ liệu duy nhất.

Công nghệ sử dụng

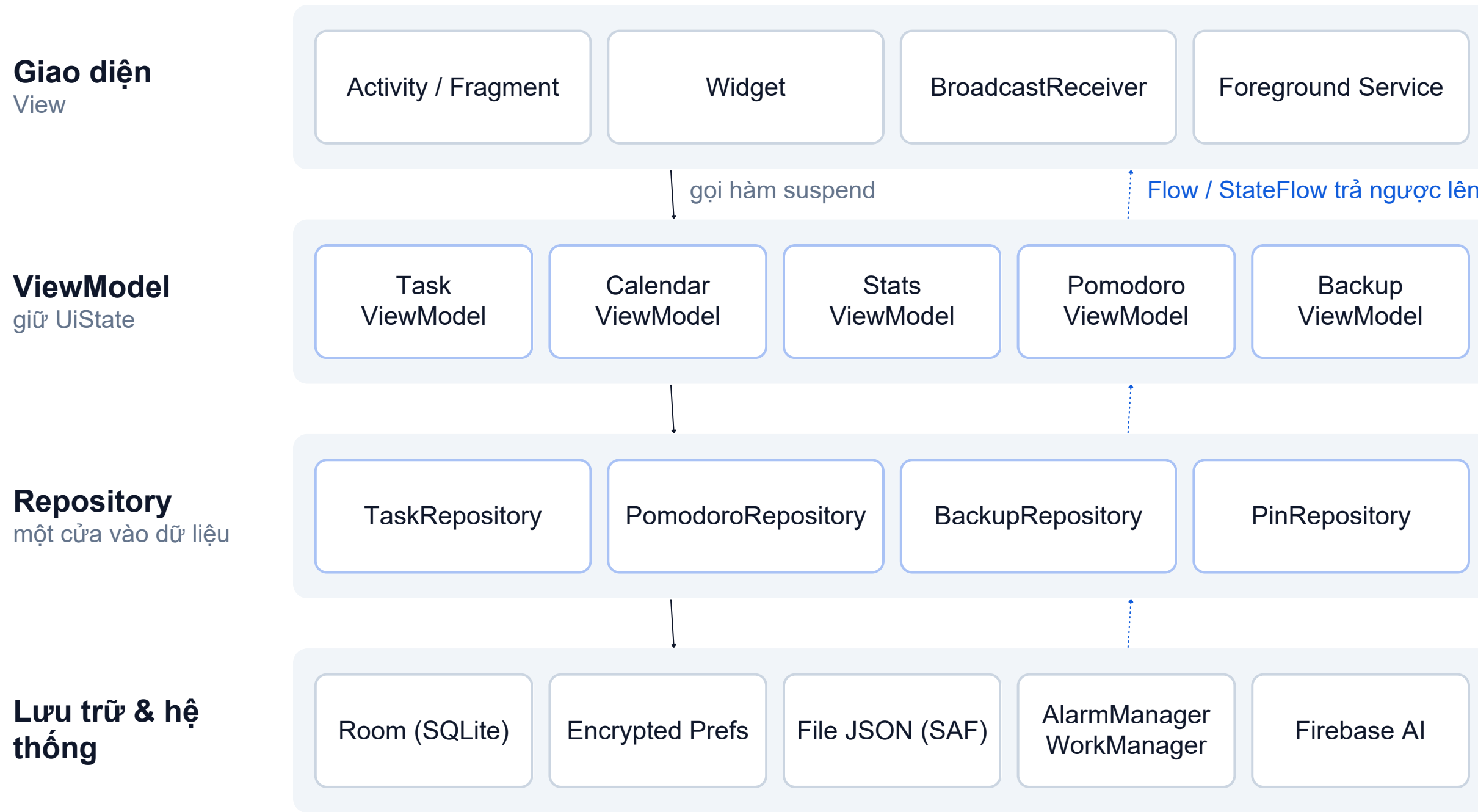
NHÓM	CÔNG NGHỆ
Ngôn ngữ & build	Kotlin 1.9 · JDK 17 · AGP 8.2 · Gradle version catalog
Nền tảng	minSdk 26 (Android 8) → targetSdk 34 (Android 14)
Kiến trúc	MVVM + Repository · Coroutines 1.7 · Flow / StateFlow
Lưu trữ	Room 2.6 (schema v4) · EncryptedSharedPreferences · JSON (Gson) qua SAF
Chạy nền	AlarmManager · WorkManager 2.9 · Foreground Service · BroadcastReceiver
Giao diện	Material 3 · Navigation 2.7 · ViewBinding · custom View · App Widget
Bảo mật	SHA-256 + salt · Android Keystore · AndroidX Biometric
AI	Firebase AI Logic (Gemini) · Firebase App Check

Package trong com.team.taskmanagementapp

PACKAGE	FILE	NỘI DUNG CHÍNH
ui/	40	Activity, Fragment, Adapter, dialog, custom View, ui/viewmodel
data/	25	entity, DAO, AppDatabase, repository, validator, model
util/	13	AlarmScheduler, TaskCompletionHelper, StreakCalculator...
pomodoro/	8	TimerEngine (thuần Kotlin), Service, CompletionTracker, config
viewmodel/	6	TaskViewModel, CalendarViewModel, ImportViewModel + factory
receiver/	5	Boot, TimeChange, Alarm, TaskNotification, StreakReminder
widget/	5	Provider, RemoteViewsService, MidnightScheduler, Updater
security/	2	PinRepository + Impl trên EncryptedSharedPreferences
ai/	1	AiTaskAssistant: dựng prompt, chuẩn hoá JSON của Gemini
worker/	1	TaskReminderWorker: dự phòng khi không có exact alarm
(gốc)	2	TaskApplication: khởi tạo DB, App Check, observer cho widget

MVVM + Repository: mỗi tầng một việc

Giao diện không bao giờ gọi thẳng database; mọi dữ liệu đi qua ViewModel và Repository.

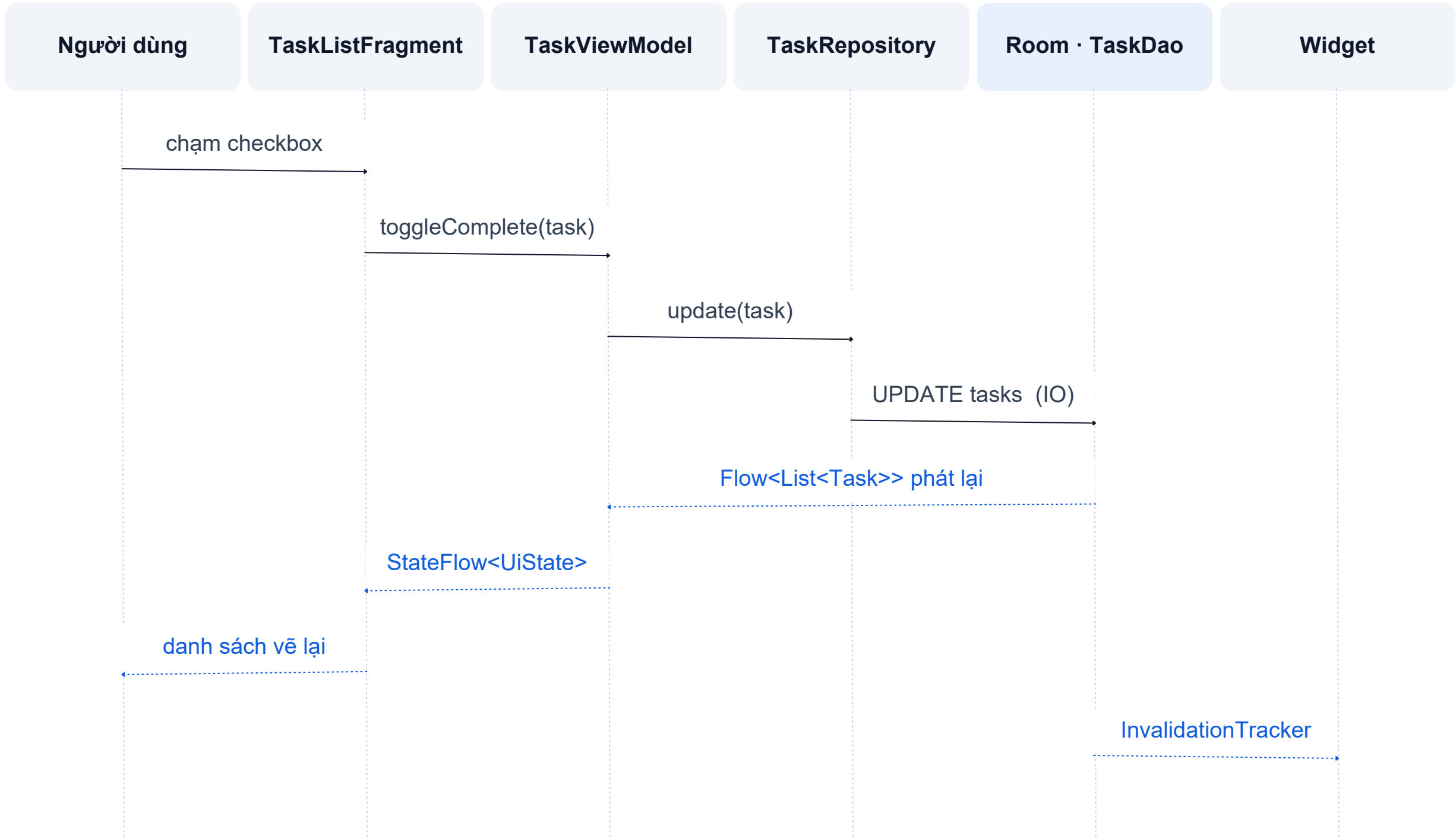


Vì sao chọn cách chia này

- 01 Làm song song**
UI không đụng DB trực tiếp, nên 5 người làm 5 tính năng trên 5 branch mà ít đụng code của nhau.
- 02 Không mất dữ liệu khi xoay**
ViewModel sống qua configuration change; form đang nhập, bộ lọc đang chọn vẫn còn.
- 03 Một thay đổi, mọi nơi cập nhật**
Room trả về Flow, nên màn hình, lịch, thống kê và widget cùng tự vẽ lại.

Tick một task: màn hình và widget tự cập nhật

Không có chỗ nào gọi “reload”. Room phát danh sách mới mỗi khi bảng tasks thay đổi.



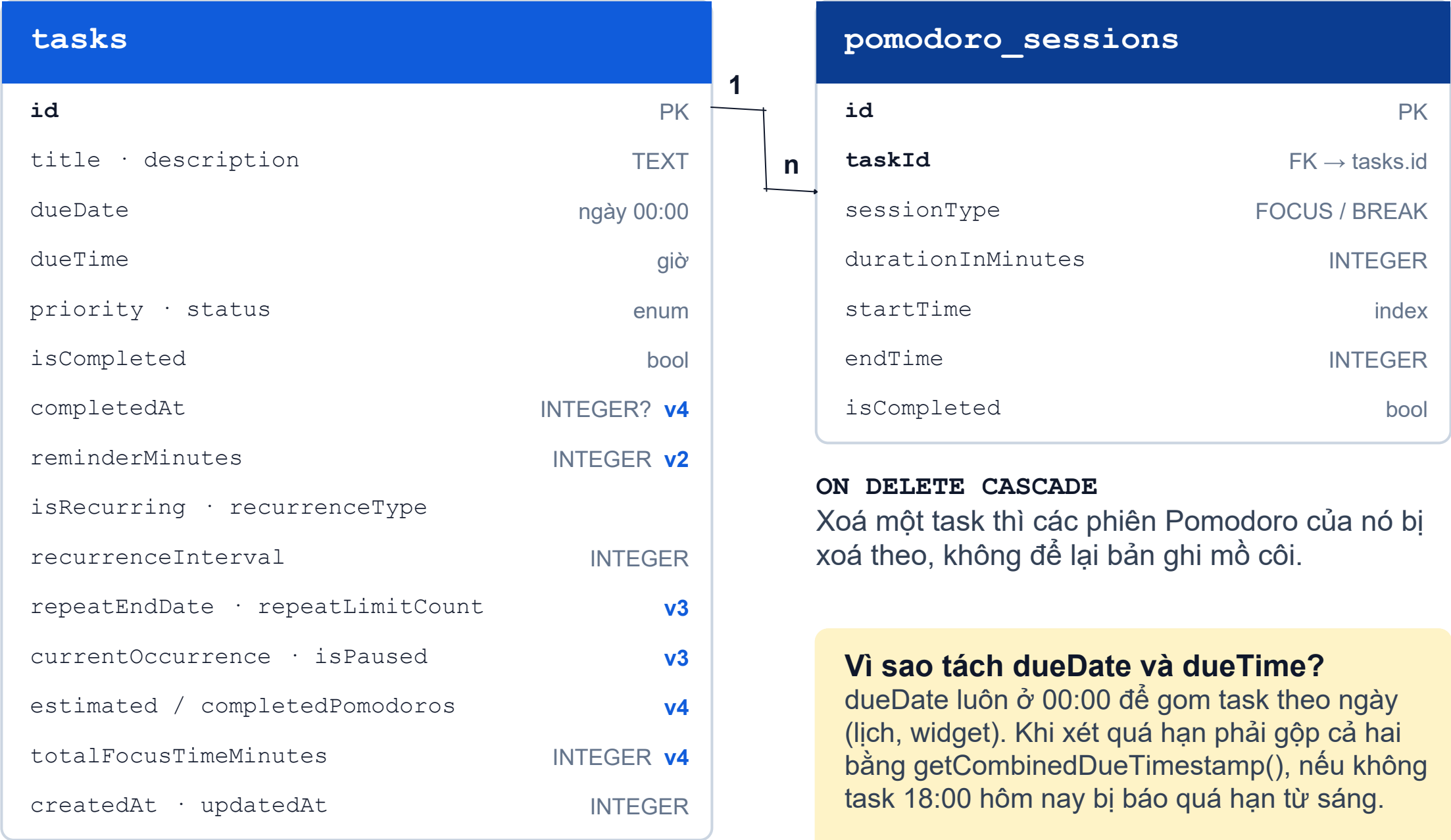
```
code thật trong repo

// TaskDao.kt
@Query("SELECT * FROM tasks
        ORDER BY dueDate ASC")
fun getAllTasks(): Flow<List<Task>>

// TaskViewModel.kt
viewModelScope.launch {
    repository.getAllTasks()
        .collect { tasks ->
            _allTasks.value = tasks
            _streakInfo.value =
                StreakCalculator
                    .calculateStreak(tasks, ..)
            _totalCompletedTasks.value =
                tasks.count { it.isCompleted }
        }
}
```

Room database, phiên bản 4, hai bảng

Mỗi lần thêm cột đều có migration riêng, nên người dùng cập nhật app không mất dữ liệu.



Lịch sử migration

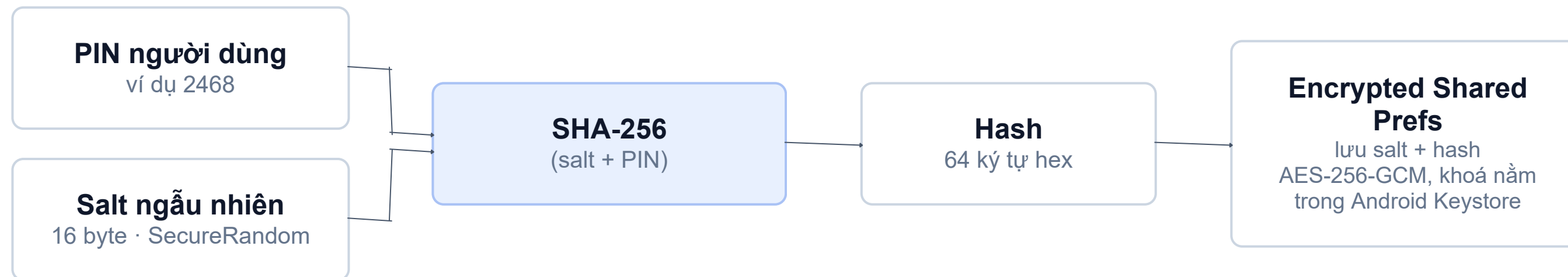
- v1** Bảng tasks ban đầu
- v2** + reminderMinutes: nhắc trước N phút
- v3** + 4 cột cho task lặp: ngày kết thúc, số lần, lần hiện tại, tạm dừng
- v4** + bảng pomodoro_sessions, completedAt và các cột thống kê Pomodoro

```
// AppDatabase.kt
db.addColumnIfMissing("tasks",
    "completedAt", "INTEGER")
// đọc PRAGMA table_info trước khi ALTER
```

Mã PIN không bao giờ được lưu dạng thô

Máy chỉ giữ salt và hash. Kể cả lấy được file cấu hình của app cũng không đọc ra được PIN.

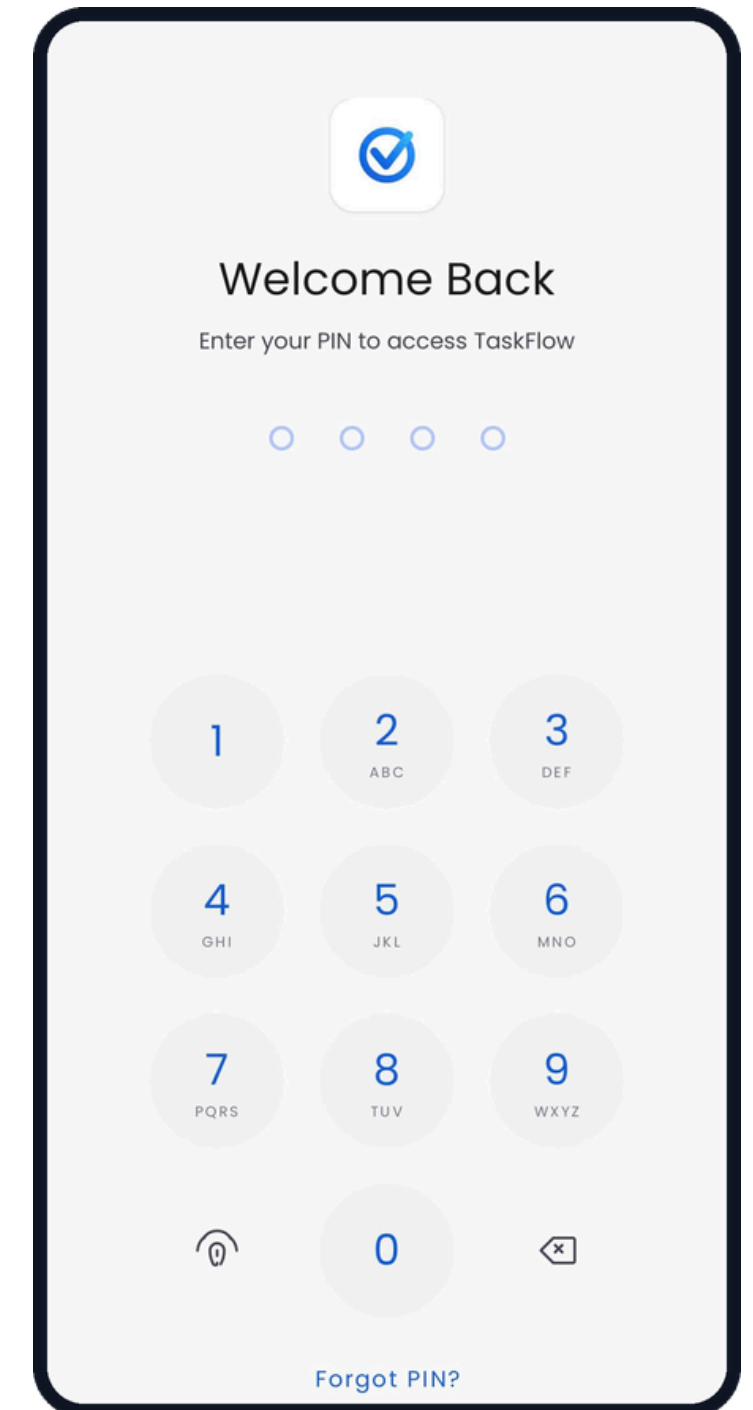
Khi đặt PIN



Khi mở khoá

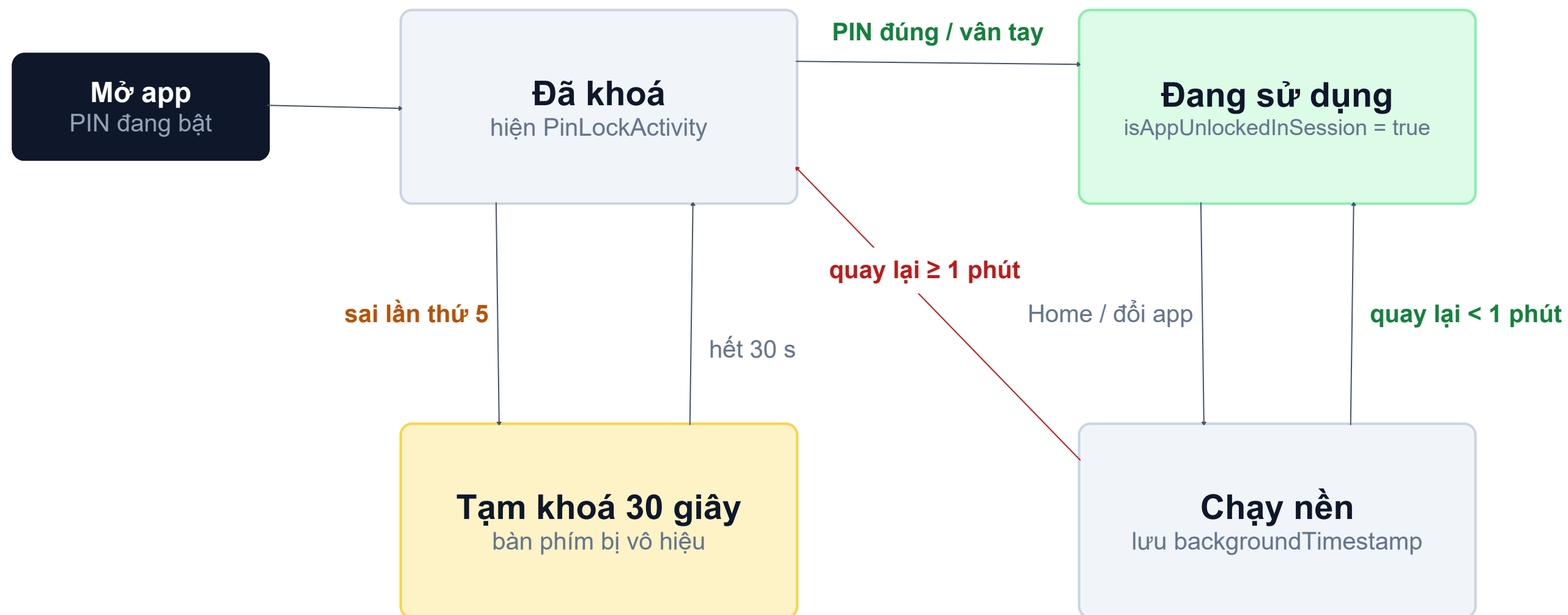


```
val input = (salt + pin).toByteArray(Charsets.UTF_8)
return MessageDigest.getInstance(HASH_ALGORITHM).digest(input).toHex() // "SHA-256"
```



Khi nào app đòi nhập lại PIN

BaseActivity theo dõi vòng đời của cả tiến trình app, không phải từng màn hình.



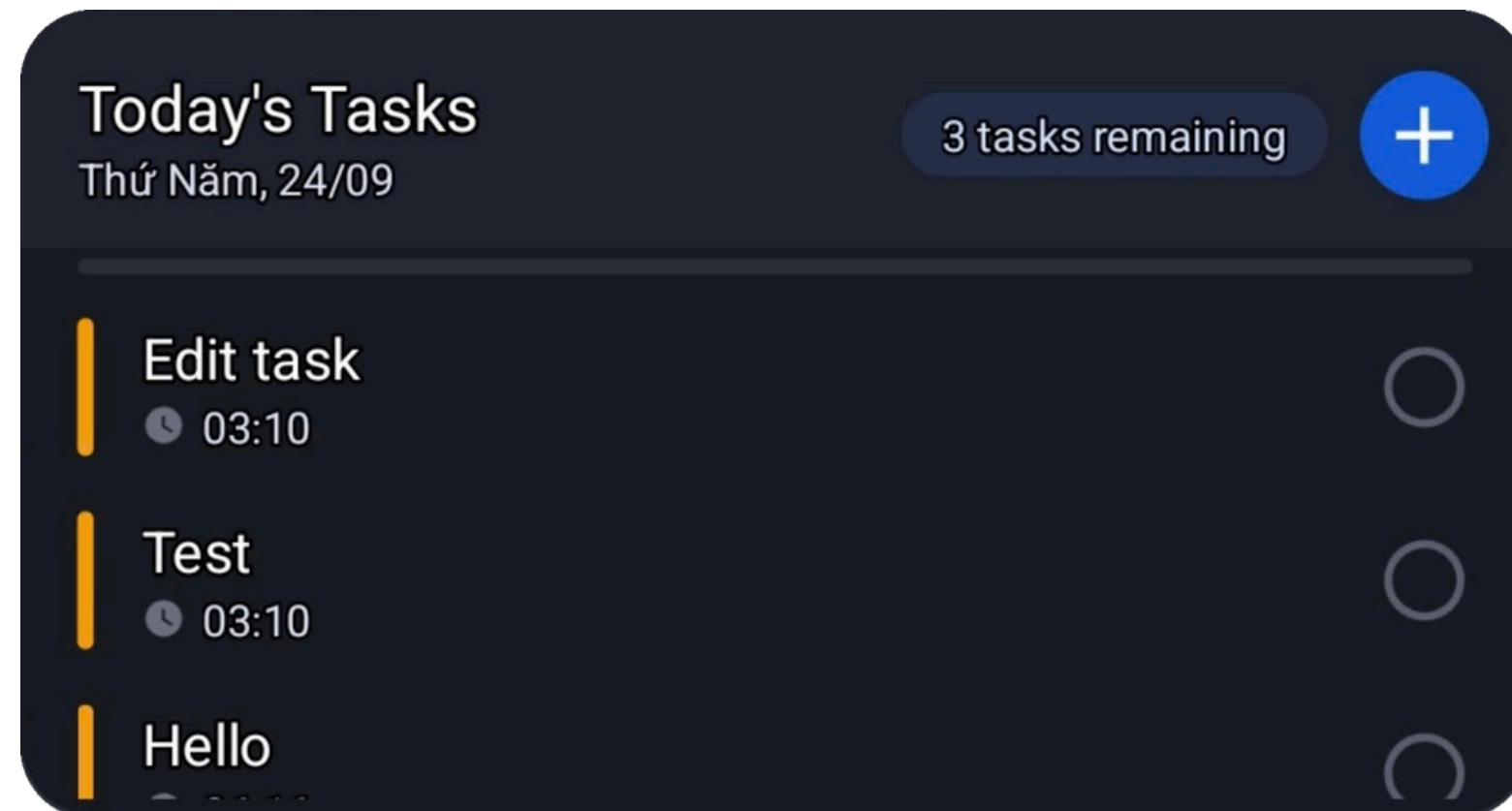
MAX_PIN_ATTEMPTS = 5 LOCKOUT_DURATION_MS = 30_000 AUTO_LOCK_TIMEOUT_MS = 60_000

Cài đặt thế nào

- **ProcessLifecycleOwner** báo khi cả app vào nền, nên chuyển qua lại giữa các màn hình không bị khoá nhầm.
- **isAppUnlockedInSession** chỉ nằm trong RAM: app bị hệ thống kill thì tự về false, mở lại phải nhập PIN.
- **Mốc hết khoá** lưu trong Encrypted Prefs: xoay màn hình hay tắt app mở lại vẫn còn bị khoá.
- **Mọi Activity kế thừa BaseActivity** nên mở từ thông báo hay shortcut cũng không lách được PIN.

Hoàn thành task ngay trên màn hình chính

Widget không giữ dữ liệu riêng. Nó đọc lại Room mỗi khi bảng tasks thay đổi, dù thay đổi đến từ đâu.



Ảnh chụp widget trên máy thật

Qua ngày mới

WidgetMidnightScheduler hẹn WorkManager đúng 00:00, lặp 24 h: danh sách tự đổi sang ngày mới dù không mở app.

Bug đã gặp

Chạm checkbox không có tác dụng. Nguyên nhân: PendingIntent template phải là FLAG_MUTABLE thì taskId mới tới được receiver.

1 Chạm vòng tròn trên widget
`PendingIntent` template + fill-in extra mang taskId

2 TaskWidgetProvider nhận broadcast
`onReceive(): EXTRA_WIDGET_ACTION = TOGGLE → goAsync()`

3 Gọi TaskCompletionHelper
`toggleById():` cập nhật Room, huỷ nhắc, xử lý task lặp

4 Room ghi xong, báo bảng tasks đã đổi
`InvalidationTracker.Observer("tasks")` trong TaskApplication

5 Widget vẽ lại
`WidgetUpdater.updateAll() → RemoteViewsFactory` đọc lại

Phần 2

Trần Văn Ngọc Thắng

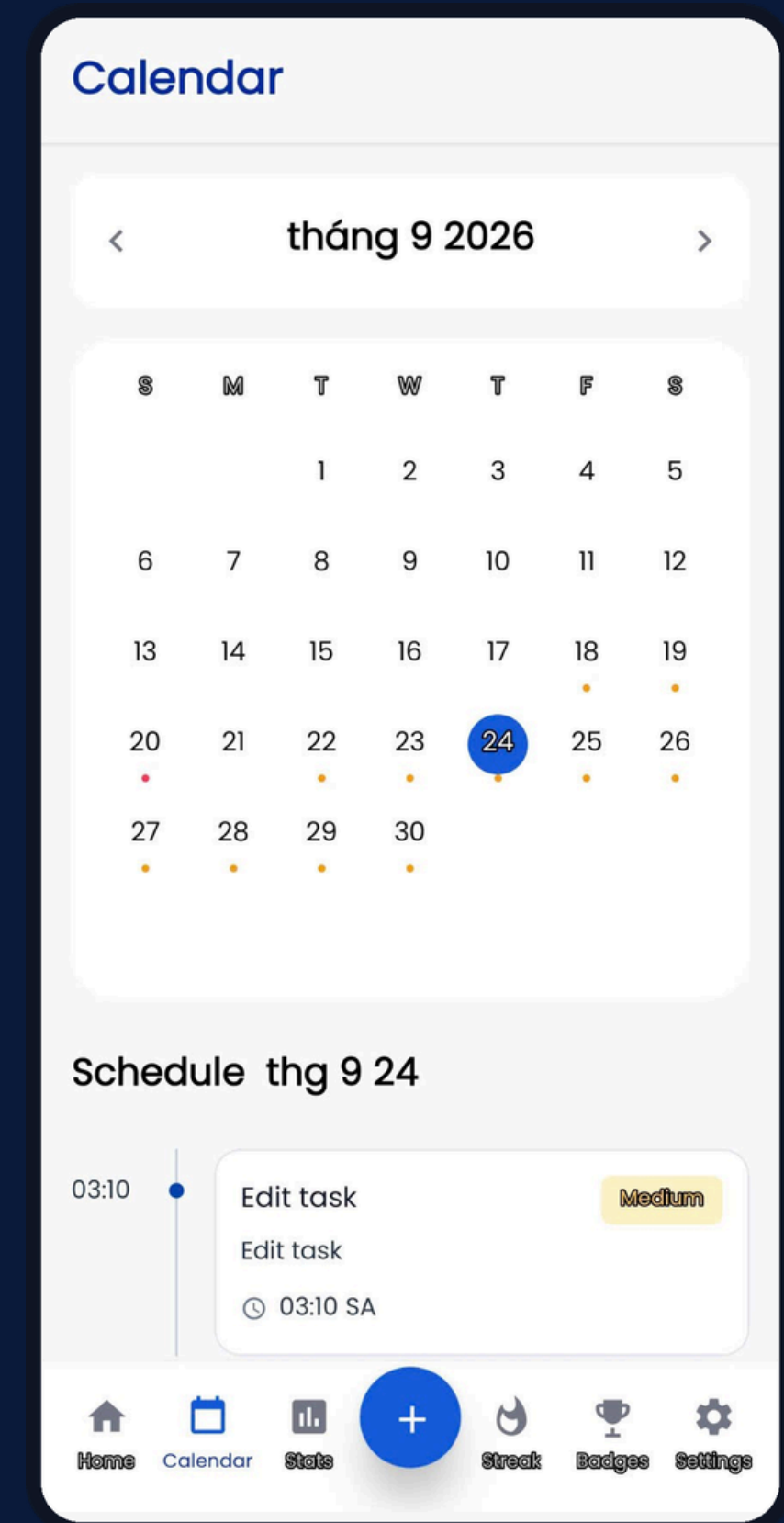
Feature Developer · Task, lịch, vân tay

14 Tạo và sửa task

15 Lịch tháng

16 Mở khoá bằng vân tay

17 App Shortcuts



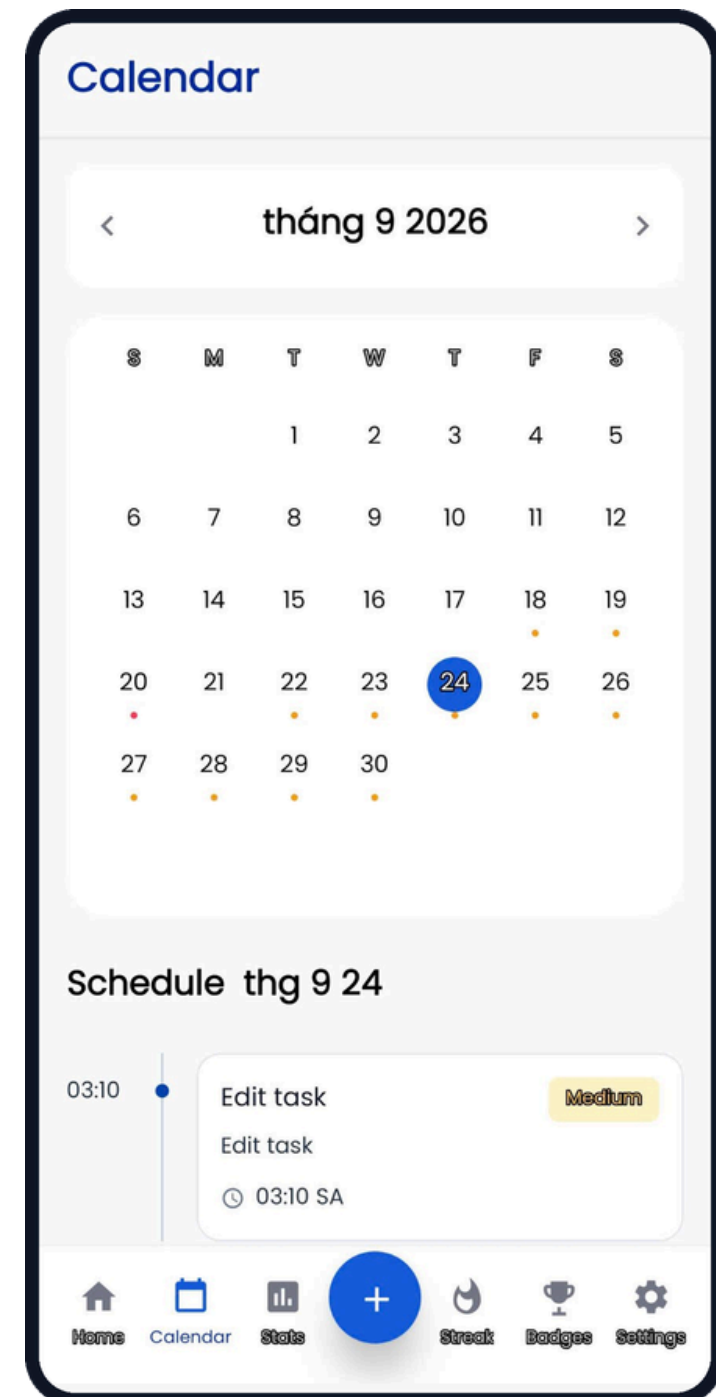
Tạo và sửa task: một màn hình, hai chế độ

AddEditTaskActivity dùng chung cho tạo mới và chỉnh sửa; khác nhau ở chỗ Intent có mang taskId hay không.



Lịch tháng: gom task theo từng ngày

CalendarViewModel giữ một bảng tra “ngày → danh sách task” cho tháng đang xem.



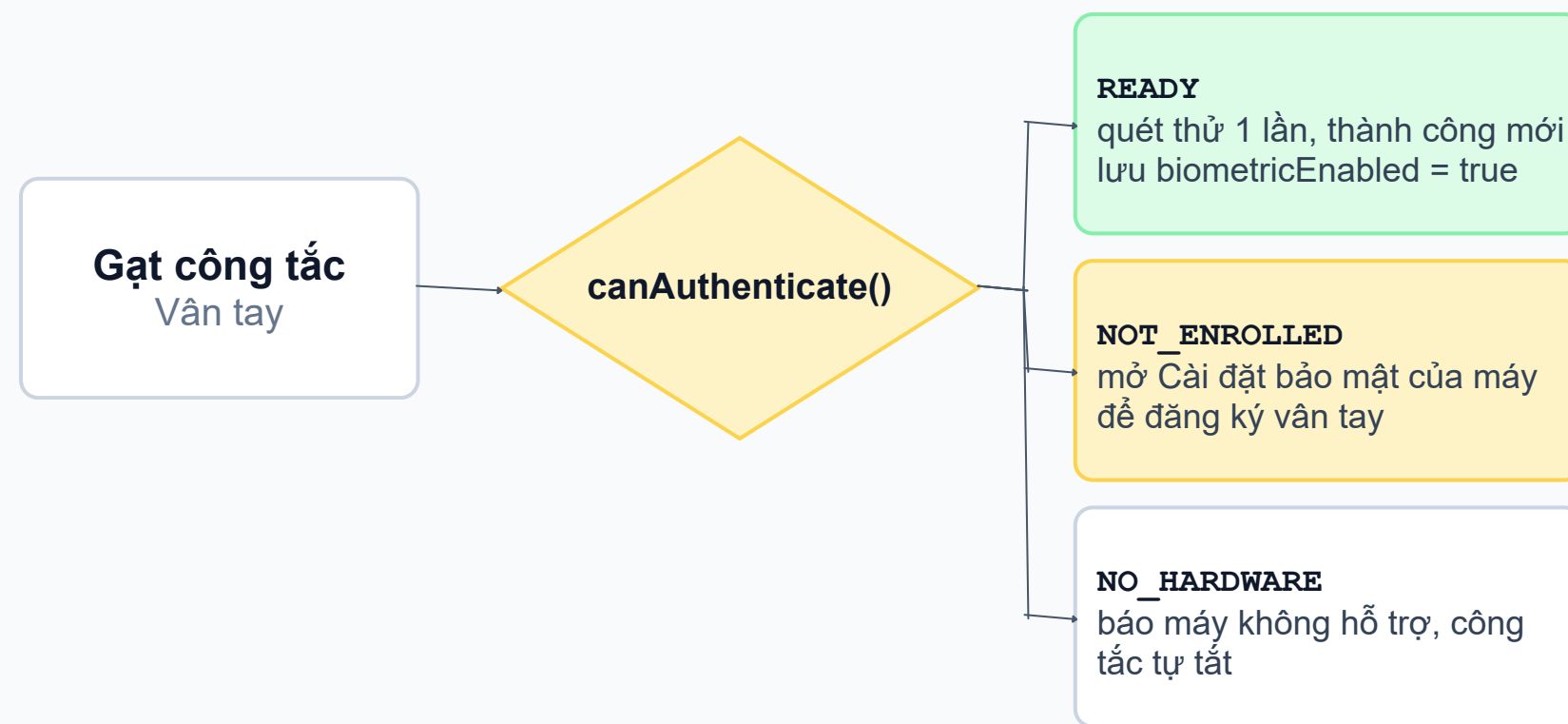
Hai chi tiết giúp lịch hiển thị đúng

- **dueDate luôn ở 00:00:** dùng thẳng làm khoá của Map, mỗi ô lịch chỉ là một lần tra thay vì duyệt cả danh sách.
- **Task lặp được “chiều” lên lịch:** task hằng ngày / tuần / tháng hiện ở mọi ngày tương ứng trong tháng, dù trong DB chỉ có một bản ghi.

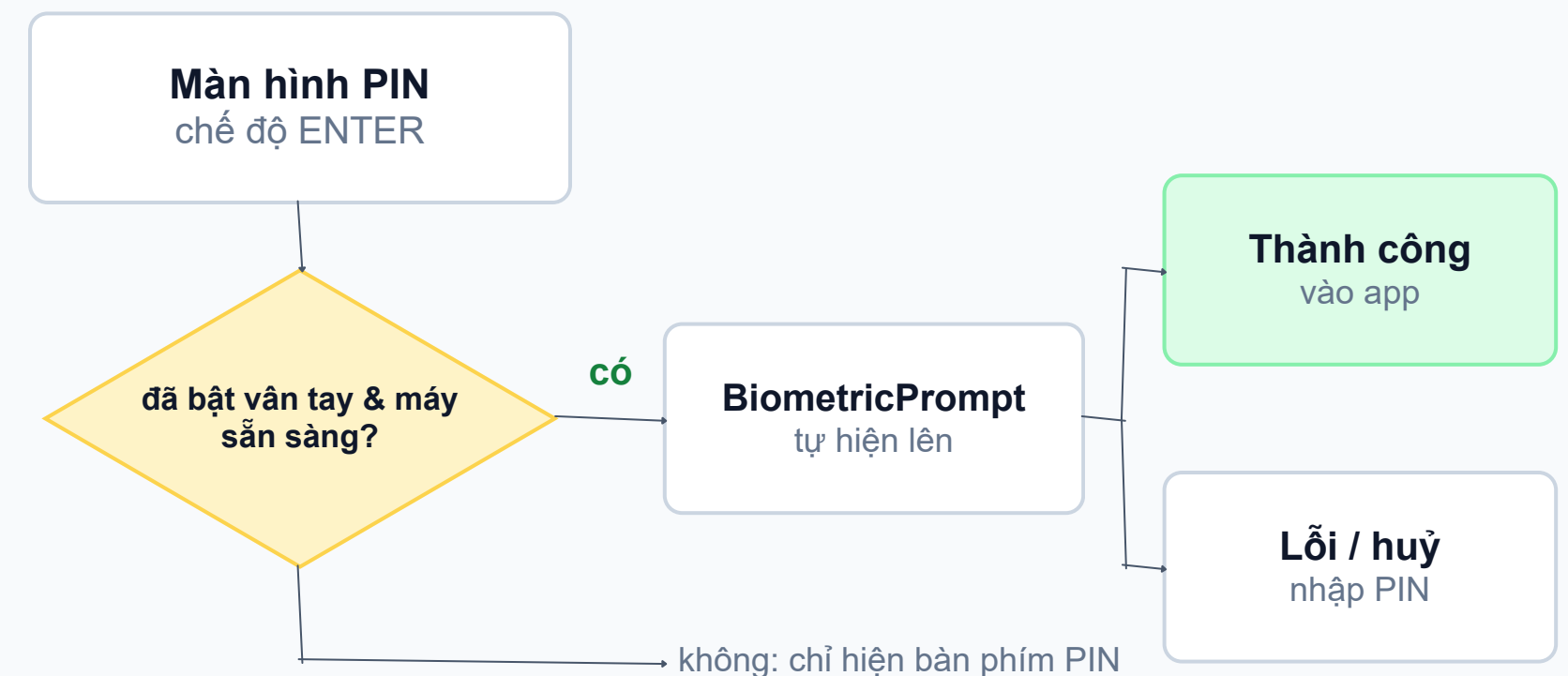
Mở khoá bằng vân tay, PIN vẫn là lớp gốc

Vân tay chỉ là lối tắt để mở khoá. Không có PIN thì không bật được vân tay.

Bật trong Cài đặt



Khi mở app



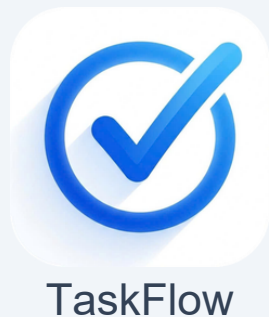
Quy tắc an toàn (có unit test)

Tắt PIN hoặc xoá PIN thì biometricEnabled luôn bị ghi về false trong cùng một lần lưu. Vân tay không thể còn hoạt động khi PIN đã tắt.

```
// PinRepositoryImpl.clearPin()
.remove(KEY_PIN_HASH).remove(KEY_PIN_SALT)
.putBoolean(KEY_PIN_ENABLED, false)
.putBoolean(KEY_BIOMETRIC_ENABLED, false)
```

App Shortcuts: nhan giu icon de vao thang chuc nang

Khai báo tĩnh trong res/xml/shortcuts.xml, không cần code tạo động.



+ Tạo task mới

☰ Việc hôm nay

📅 Lịch

📊 Thống kê

shortcuts.xml

4 thẻ <shortcut>, mỗi thẻ một Intent

Intent tới MainActivity

action = ...SHORTCUT_TODAY (ví dụ)

handleShortcutIntent()

gọi ở onCreate và onNewIntent

BaseActivity.onResume()

PIN bật mà chưa mở khoá → màn PIN trước

Điều hướng

navController.navigate(...) hoặc mở AddEditTaskActivity

res/xml/shortcuts.xml

```
<shortcut
    android:shortcutId="shortcut_today"
    android:icon=
        "@drawable/ic_shortcut_today"
    android:shortcutShortLabel=
        "@string/shortcut_today_short">
    <intent
        android:action=
            "...action.SHORTCUT_TODAY"
        android:targetClass=
            "...MainActivity" />
    </shortcut>
```

Lỗi hổng đã vá

Shortcut “Tạo task” mở AddEditTaskActivity ngay lập tức. Ban đầu màn này chưa kế thừa BaseActivity nên lọt qua được PIN; đã sửa để kế thừa.

Phần 3

Nguyễn Lê Huy Tâm

Feature Developer · Task lập, import, streak

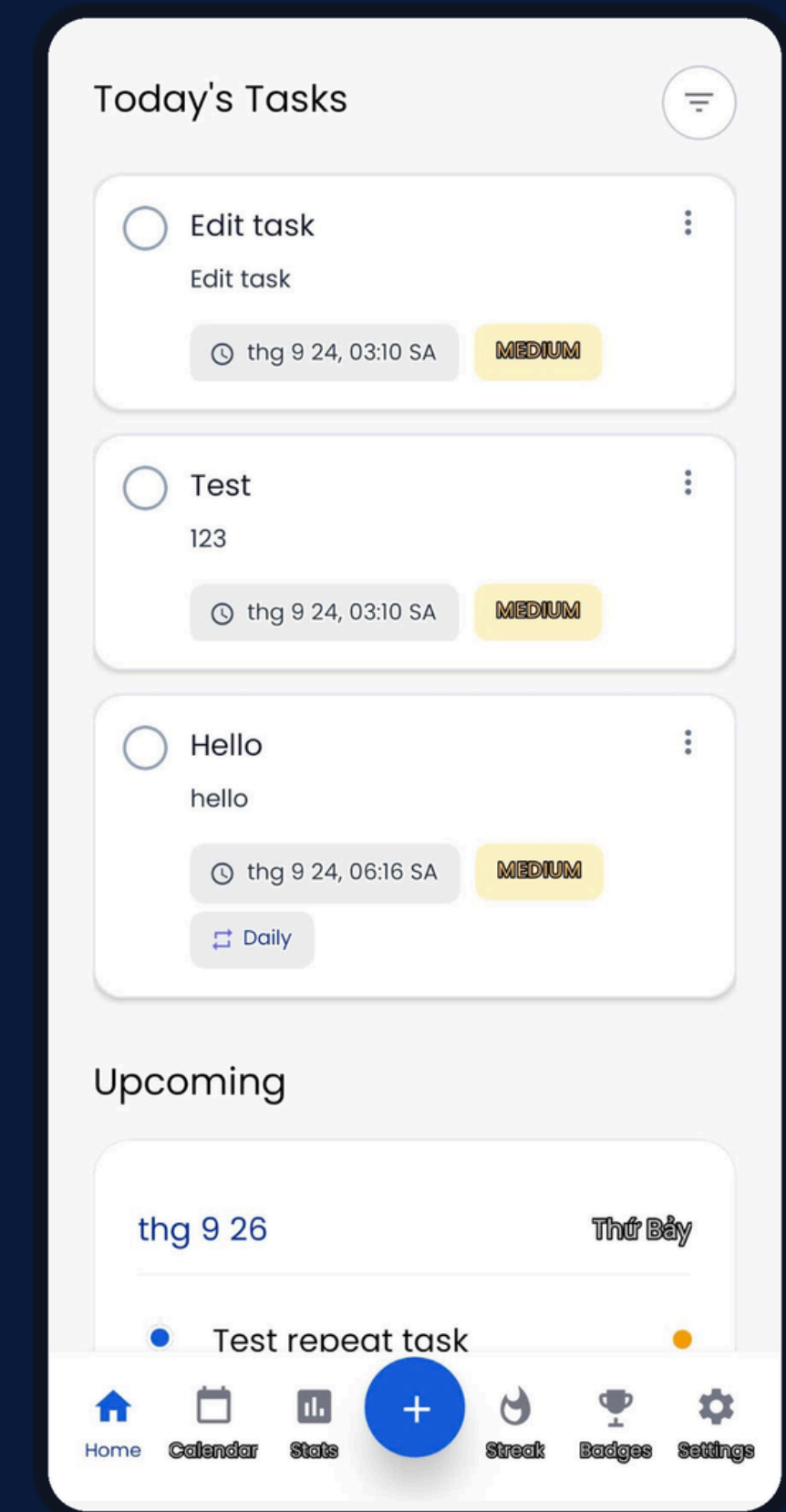
19 Task lập lại: cấu hình và cách tính ngày

20 Khi hoàn thành một task lập

21 Import JSON có kiểm tra

22 Streak: chuỗi ngày hoàn thành

23 7 huy hiệu đặc biệt



Task lặp lại: 4 chu kỳ, 3 cách kết thúc

Tại mỗi thời điểm, một chuỗi chỉ có một bản nằm ở tương lai; bản kế tiếp sinh ra khi bản hiện tại được hoàn thành.

Chu kỳ

Hàng ngày

Hàng tuần

Hàng tháng

Hàng năm

recurrenceInterval: lặp mỗi N kỳ, ví dụ 2 tuần một lần

Kết thúc

Không bao giờ

Đến ngày...

Sau N lần

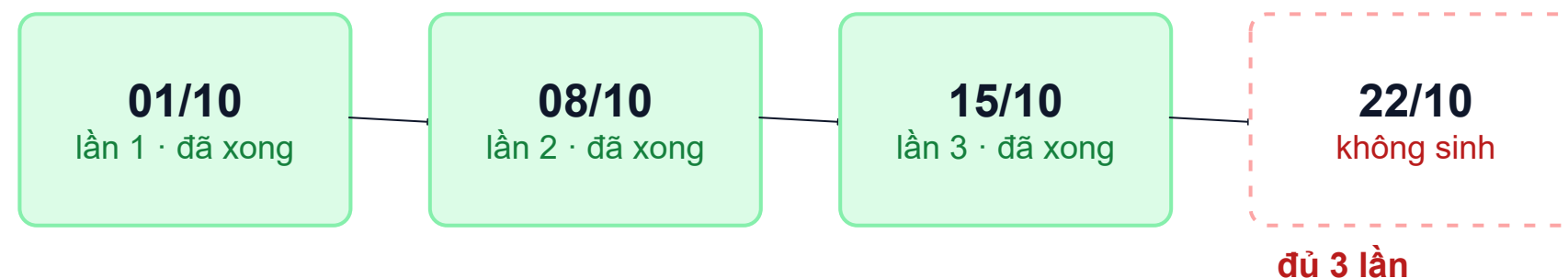
repeatEndDate · repeatLimitCount + currentOccurrence

Tạm dừng

Tạm dừng / Tiếp tục

isPaused: giữ chuỗi, không sinh bản mới

Ví dụ: “Nộp nhật ký thực tập” hàng tuần, kết thúc sau 3 lần



Vì sao cộng ngày bằng LocalDate?

PHÉP TÍNH	CỘNG MILI-GIÂY	<code>LocalDate.plusMonths(1)</code>
31/01/2026 + 1 tháng	02/03/2026 ✗	28/02/2026 ✓
31/01/2028 + 1 tháng	01/03/2028 ✗	29/02/2028 ✓ (năm nhuận)

```
RecurrenceHelper.calculateNextDueDate() (n = interval)

val currentLocalDate = Instant.ofEpochMilli(dueDateMillis)
    .atZone(zoneId).toLocalDate()
val nextLocalDate = when (type) {
    RecurrenceType.DAILY -> currentLocalDate.plusDays(n)
    RecurrenceType.WEEKLY -> currentLocalDate.plusWeeks(n)
    RecurrenceType.MONTHLY -> currentLocalDate.plusMonths(n)
    RecurrenceType.YEARLY -> currentLocalDate.plusYears(n)
}
```

Migration v2 → v3 thêm 4 cột repeatEndDate, repeatLimitCount, currentOccurrence, isPaused; enum RecurrenceType thêm YEARLY.

Khi tick hoàn thành một task lặp

Widget, thông báo và trợ lý AI gọi TaskCompletionHelper; màn hình chính có bản xử lý tương đương trong TaskViewModel.



Bỏ tick (hoàn tác): xóa bản tương lai vừa sinh và hủy nhắc của nó, chuỗi trở về như trước khi tick.

```

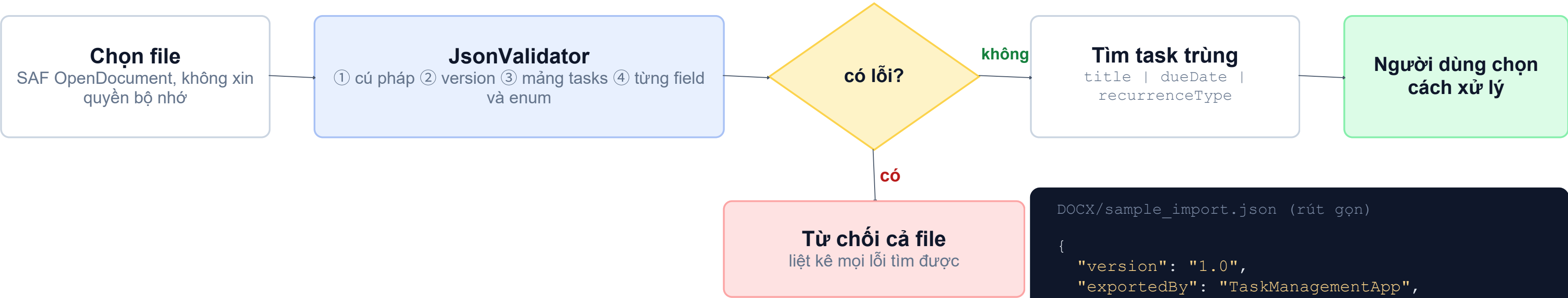
RecurrenceHelper.kt (rút gọn)

fun isRecurrenceEnded(
    task: Task,
    nextDueDateMillis: Long
): Boolean {
    if (task.isPaused) return true
    if (task.repeatEndDate > 0L &&
        nextDueDateMillis >
            task.repeatEndDate)
        return true
    if (task.repeatLimitCount > 0 &&
        task.currentOccurrence >=
            task.repeatLimitCount)
        return true
    return false
}
  
```

Bản tương lai được tìm theo tiêu đề + chu kỳ bằng `getFutureRecurringTasksSync()`, nên tick hai lần liên tiếp cũng không sinh hai bản.

Import JSON: kiểm tra toàn bộ file rồi mới ghi

Một lỗi là cả file bị từ chối, và người dùng thấy toàn bộ danh sách lỗi chứ không chỉ lỗi đầu tiên.



3 cách xử lý task trùng (ConflictAction)

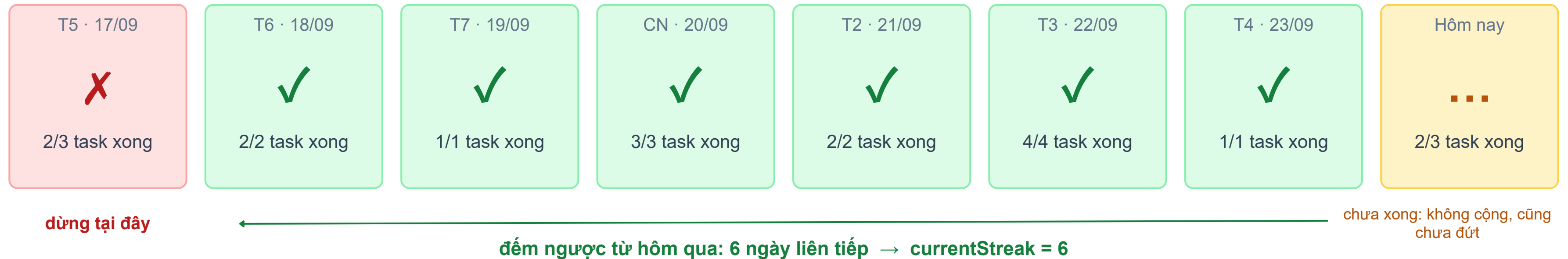
CÁCH	TASK BỊ TRÙNG	TASK CHƯA CÓ
SKIP (mặc định)	giữ bản đang có, bỏ bản trong file	thêm vào
REPLACE	xoá bản đang có, thêm bản trong file	thêm vào
REPLACE_ALL	xoá toàn bộ task hiện có rồi import lại cả file	

```
DOCX/sample_import.json (rút gọn)

{
  "version": "1.0",
  "exportedBy": "TaskManagementApp",
  "tasks": [
    {
      "title": "Buy groceries",
      "dueDate": 1788714000000,
      "priority": "MEDIUM",
      "status": "TODO",
      "recurrenceType": "NONE",
      "reminderMinutes": 30
    }, ...
  ]
}
```

Streak: đếm ngược từ hôm nay

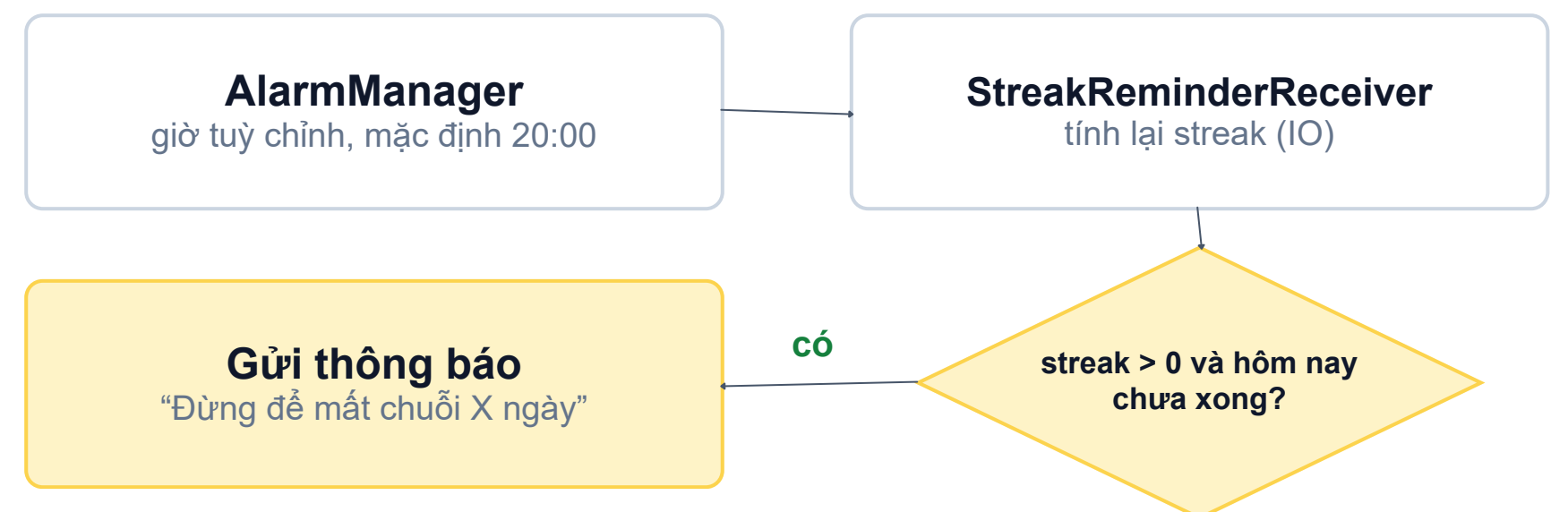
Một ngày chỉ được tính khi đã hoàn thành hết task của ngày đó.



Quy tắc trong StreakCalculator

- Ngày không có task nào cũng làm đứt chuỗi.
- Hôm nay xong 100 % thì được cộng thêm 1.
- Best streak lưu trong SharedPreferences, chỉ tăng.
- Mốc thưởng chuỗi: 3 · 7 · 30 · 50 · 100 · 200 · 365 ngày. Mốc số task: 10 → 2 000.

Nhắc giữ chuỗi mỗi tối



7 huy hiệu đặc biệt và điều kiện mở khoá

SpecialBadgeCalculator duyệt các task đã hoàn thành và kiểm tra từng điều kiện dưới đây.

HUY HIỆU	ĐIỀU KIỆN MỞ KHOÁ	CÁCH KIỂM TRA TRONG CODE
Hard Worker	Hoàn thành 10 task ưu tiên High hoặc Urgent	đếm task đã xong có priority HIGH / URGENT
Early Bird	50 task hoàn thành trước 9 giờ sáng	giờ của completedAt < 9
Night Owl	50 task hoàn thành từ 21 giờ trở đi	giờ của completedAt ≥ 21
Month Conqueror	Trọn một tháng, ngày nào cũng xong ít nhất 1 task	số ngày khác nhau có task xong = số ngày của tháng
Unstoppable	100 task trong 7 ngày liên tiếp	cửa sổ trượt 7 ngày, tổng ≥ 100
Comeback Kid	Từng mất chuỗi, sau đó đạt lại chuỗi ≥ 7 ngày	so current streak với best streak
Ultimate Master	Chuỗi 365 ngày và 2 000 task hoàn thành	streak ≥ 365 và tổng task xong ≥ 2 000

Phần 4

Huỳnh Đình Chấn

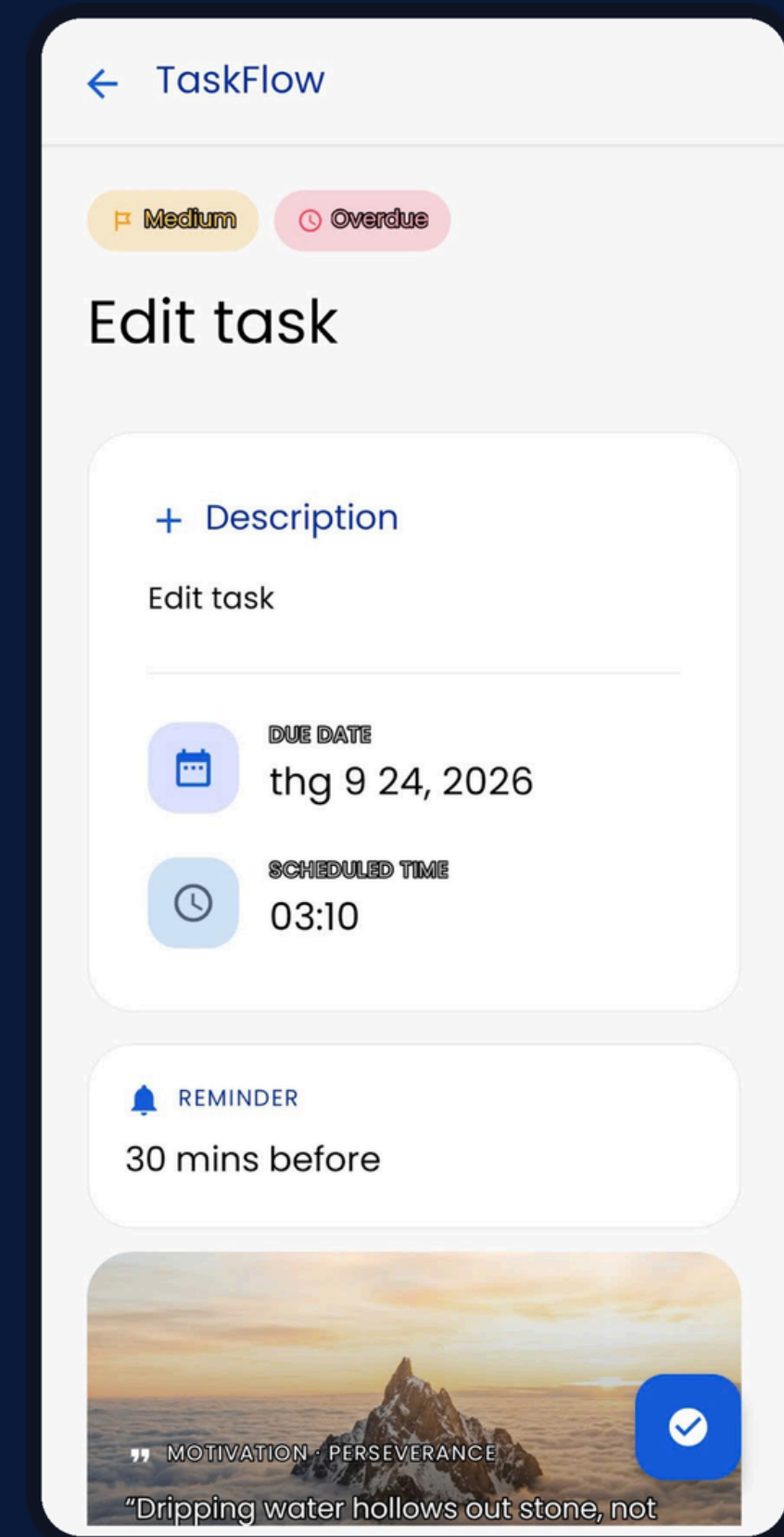
Feature Developer · Pomodoro, độ bền dữ liệu

25 Pomodoro: một máy trạng thái

26 Pomodoro chạy nền không trôi giờ

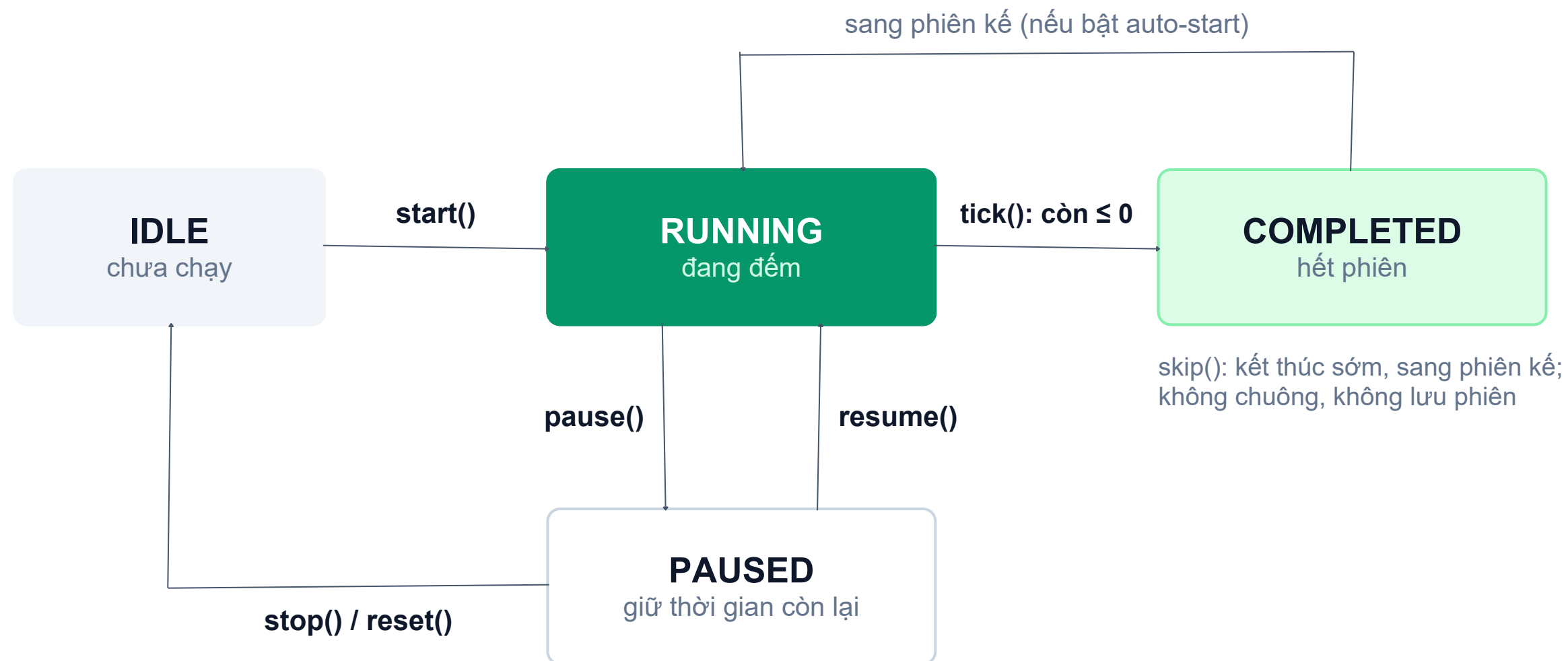
27 Xoá, quá hạn và đổi giờ

28 Backup & restore an toàn

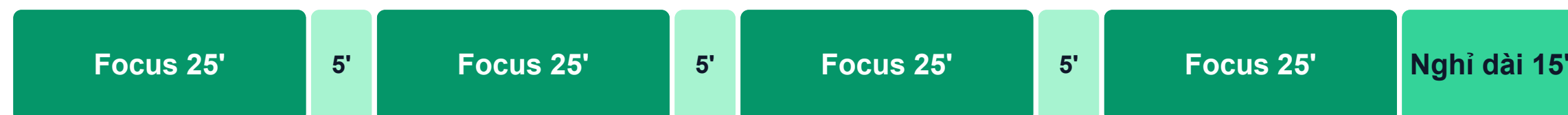


Pomodoro: một máy trạng thái thuần Kotlin

PomodoroTimerEngine không phụ thuộc Android, nên 34 unit test chạy thẳng trên JVM với đồng hồ giả.



Một đợt mặc định (cyclesBeforeLongBreak = 4)



Vì sao tách engine khỏi Service

```

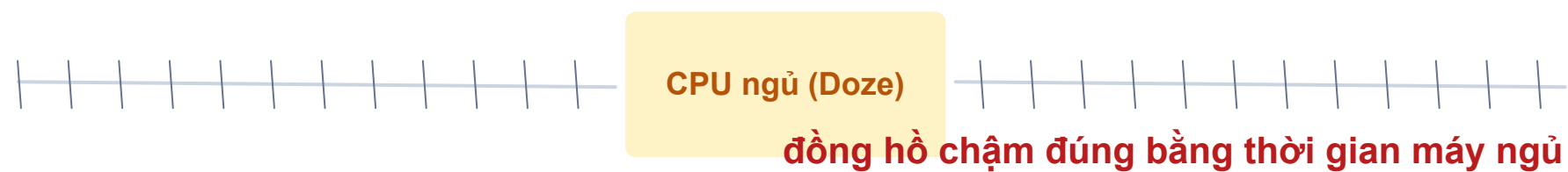
class PomodoroTimerEngine(
    initialConfig: PomodoroConfig =
        PomodoroConfig(),
    private val elapsedTimeProvider:
        () -> Long =
            { SystemClock.elapsedRealtime() },
    private val wallClockProvider:
        () -> Long =
            { System.currentTimeMillis() }
)
  
```

- Test truyền đồng hồ giả vào elapsedTimeProvider, “tua” 25 phút trong tích tắc.
- Engine đơn luồng, chỉ Service gọi, nên không cần synchronized.
- nextSessionTypeAfter(): Focus → nghỉ ngắn; đủ 4 Focus → nghỉ dài; nghỉ xong → Focus.

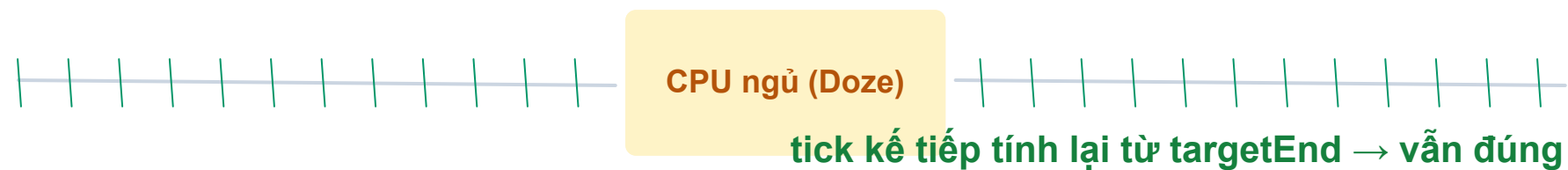
Chạy nền mà không trôi giờ

Không trừ dần từng giây. Mỗi lần tick, thời gian còn lại được tính lại từ một mốc kết thúc cố định.

Cách thường `remaining -= 1000` mỗi tick



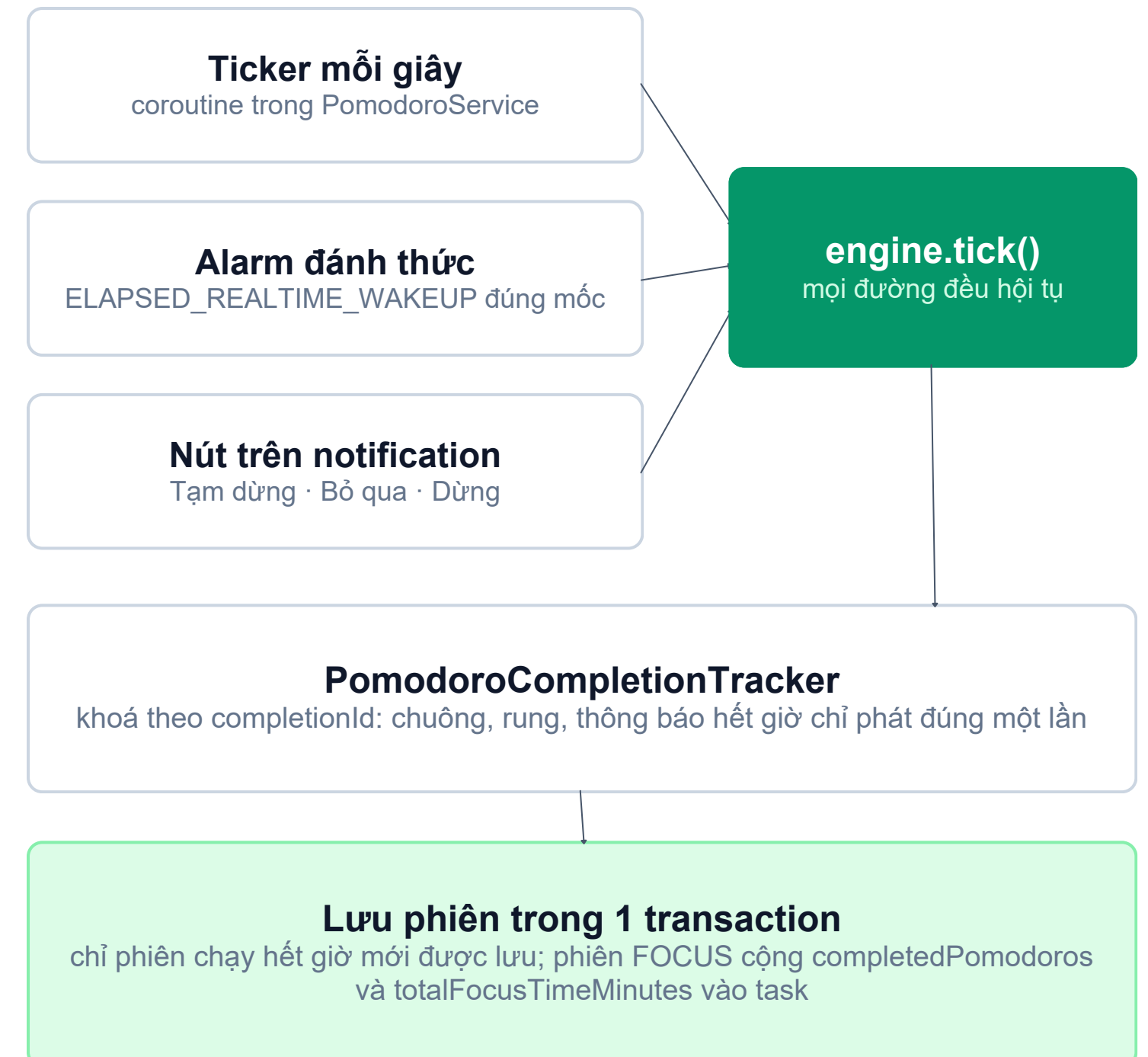
TaskFlow `remaining = targetEnd - elapsedRealtime()`



```
// beginSession(): chốt mốc kết thúc một lần
targetEndElapsedRealtime = now + total

// PomodoroSnapshot.remainingMillisAt()
(targetEndElapsedRealtime - nowElapsedRealtime).coerceAtLeast(0L)

// tick()
if (remaining <= 0L) finishSession(completed = true)
```



Giữ task, nhắc nhở và thời gian luôn khớp nhau

Ba tình huống dễ sinh lỗi “ma” và cách app xử lý từng tình huống.

Xoá task

Lỗi cần tránh: xoá rồi mà điện thoại vẫn đổ chuông

Hộp thoại xác nhận

`DialogFragment`, còn nguyên khi xoay màn hình

Xoá khỏi Room

task lập có lựa chọn xoá riêng

Hủy nhắc

`cancelAlarm()`: alarm + việc `WorkManager`

Gỡ thông báo đang hiện

`NotificationHelper.cancelNotification()`

Quá hạn

Lỗi cần tránh: task 18:00 hôm nay bị báo quá hạn từ sáng

Gộp ngày và giờ

`getCombinedDueTimestamp(dueDate, dueTime)`

So với hiện tại

DAO: `dueDate < now` và chưa hoàn thành

Chuyển sang OVERDUE

badge đỏ, xuất hiện ở bộ lọc Quá hạn

Sửa hạn sang tương lai

tự trả về trạng thái cũ (7 unit test)

Kiểm tra lại khi mở app, sau reboot và khi đổi giờ.

Đổi giờ / múi giờ

Lỗi cần tránh: đổi múi giờ làm nhắc lệch giờ

Hệ thống phát broadcast

`TIME_CHANGED · TIMEZONE_CHANGED · DATE_CHANGED`

TimeChangeReceiver

`goAsync()` + `coroutine Dispatchers.IO`

Đặt lại mọi nhắc

cho các task chưa hoàn thành

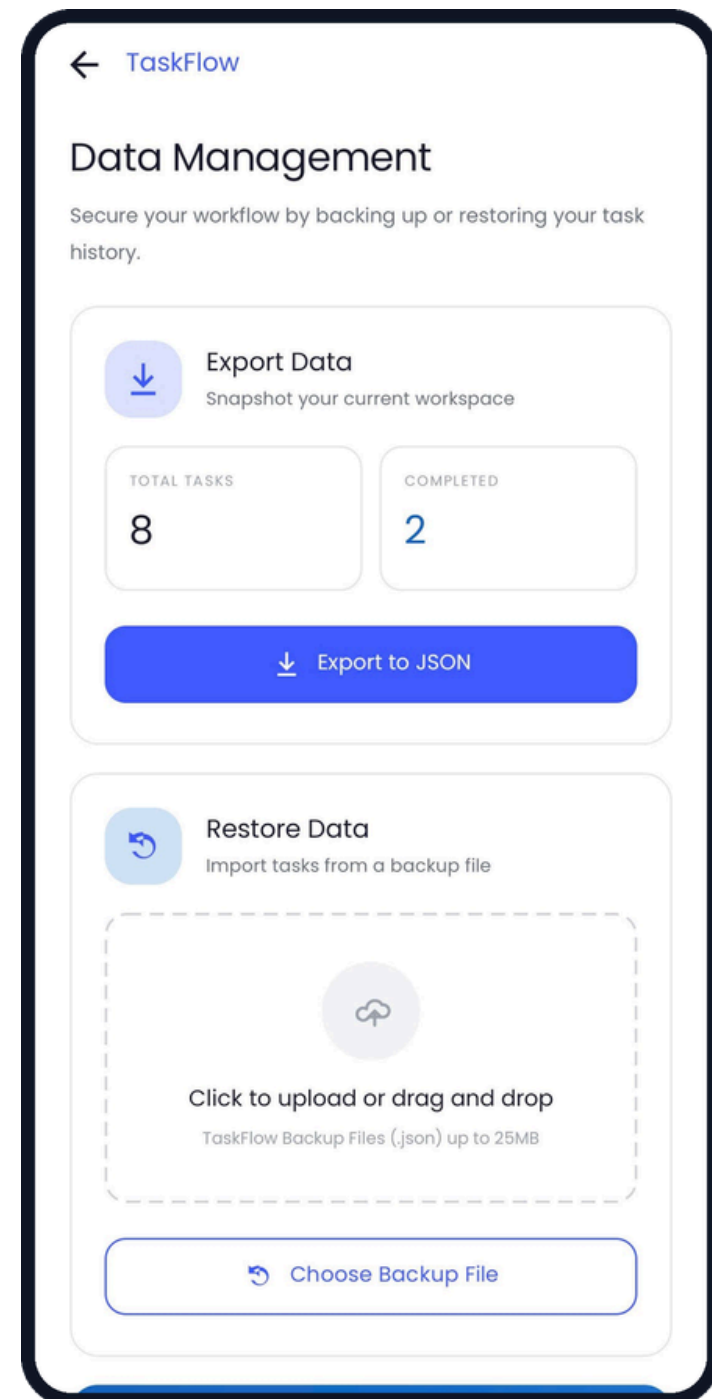
Cập nhật quá hạn, làm mới widget

`WidgetUpdater.updateAllSafe()`

`BootReceiver` làm tương tự sau khi khởi động lại máy.

Backup day du, restore theo kiểu "tất cả hoặc không"

File backup chứa cả task lẫn lịch sử Pomodoro. Restore lỗi giữa chừng thì dữ liệu cũ vẫn còn nguyên.

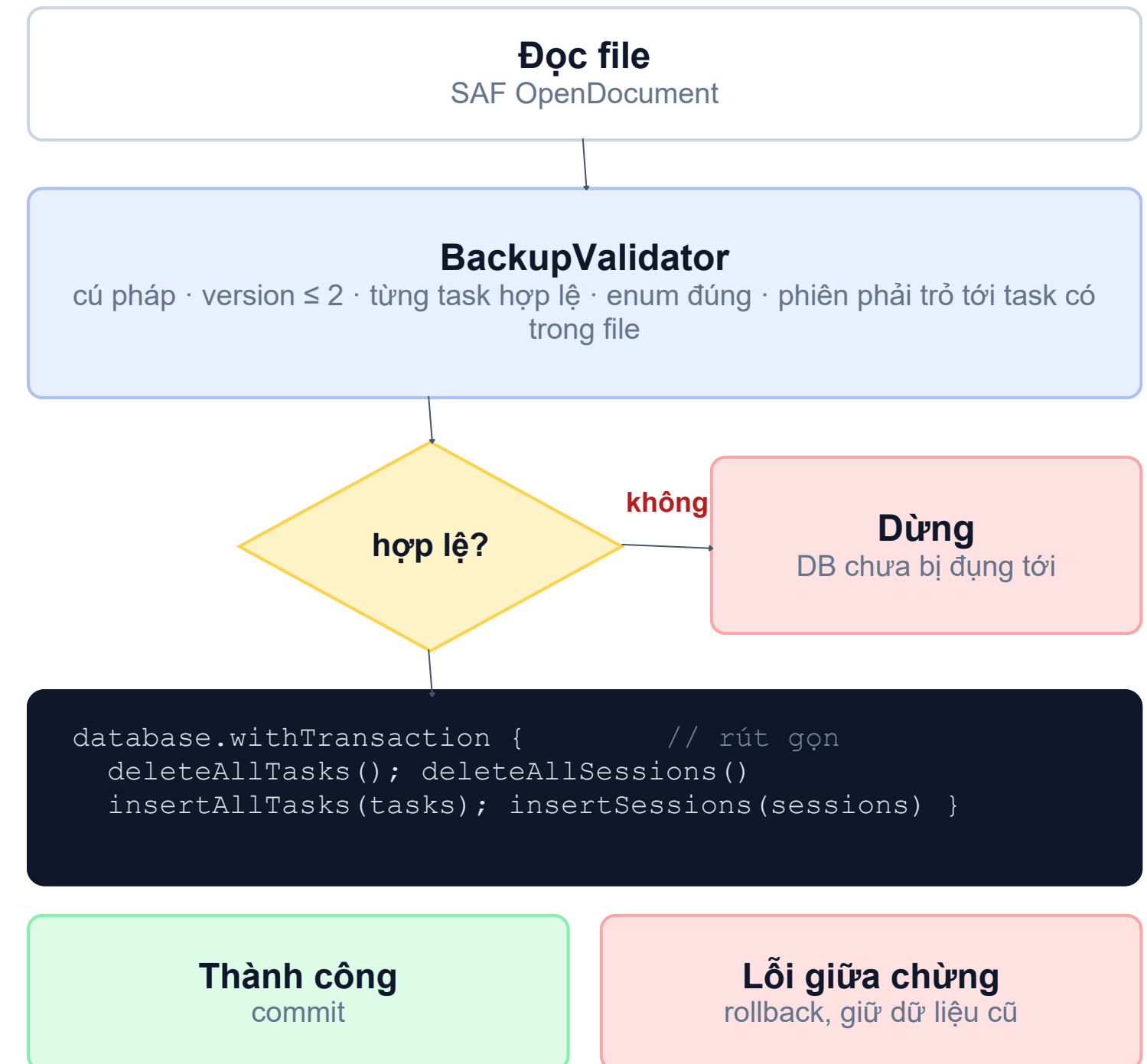


cấu trúc file backup (ExportData)

```
{
  "version": 2,
  "exportDate": "...",
  "taskCount": 12,
  "tasks": [ { ... } ],
  "pomodoroSessionCount": 30,
  "pomodoroSessions": [
    { "taskId": 3,
      "sessionType": "FOCUS",
      "durationInMinutes": 25, ... }
  ]
}
```

Export

SAF CreateDocument, tên gợi ý taskflow_backup_yyyyMMdd_HH:mm:ss.json. Người dùng chọn lưu vào máy hay Google Drive.



Phần 5

Nguyễn Ngọc Gia Bảo

QA & Feature Developer · Nhắc nhở, lọc, AI

- 30 Nhắc nhở đúng giờ
- 31 Bộ lọc và kiểm tra dữ liệu nhập
- 32 Trợ lý AI: luồng xử lý
- 33 Trợ lý AI: lệnh và an toàn
- 34 Kiểm thử

Filter & Sort

[Clear all](#)

Find exactly the tasks you need

COMPLETION

AllNot doneDone

STATUS Select multiple

TodoIn ProgressOverdue

PRIORITY Select multiple

UrgentHighMediumLow

DUE DATE

✓ AllOverdueToday

Next 7 daysThis MonthNo due date

Custom range

SORT BY

Sort tasks

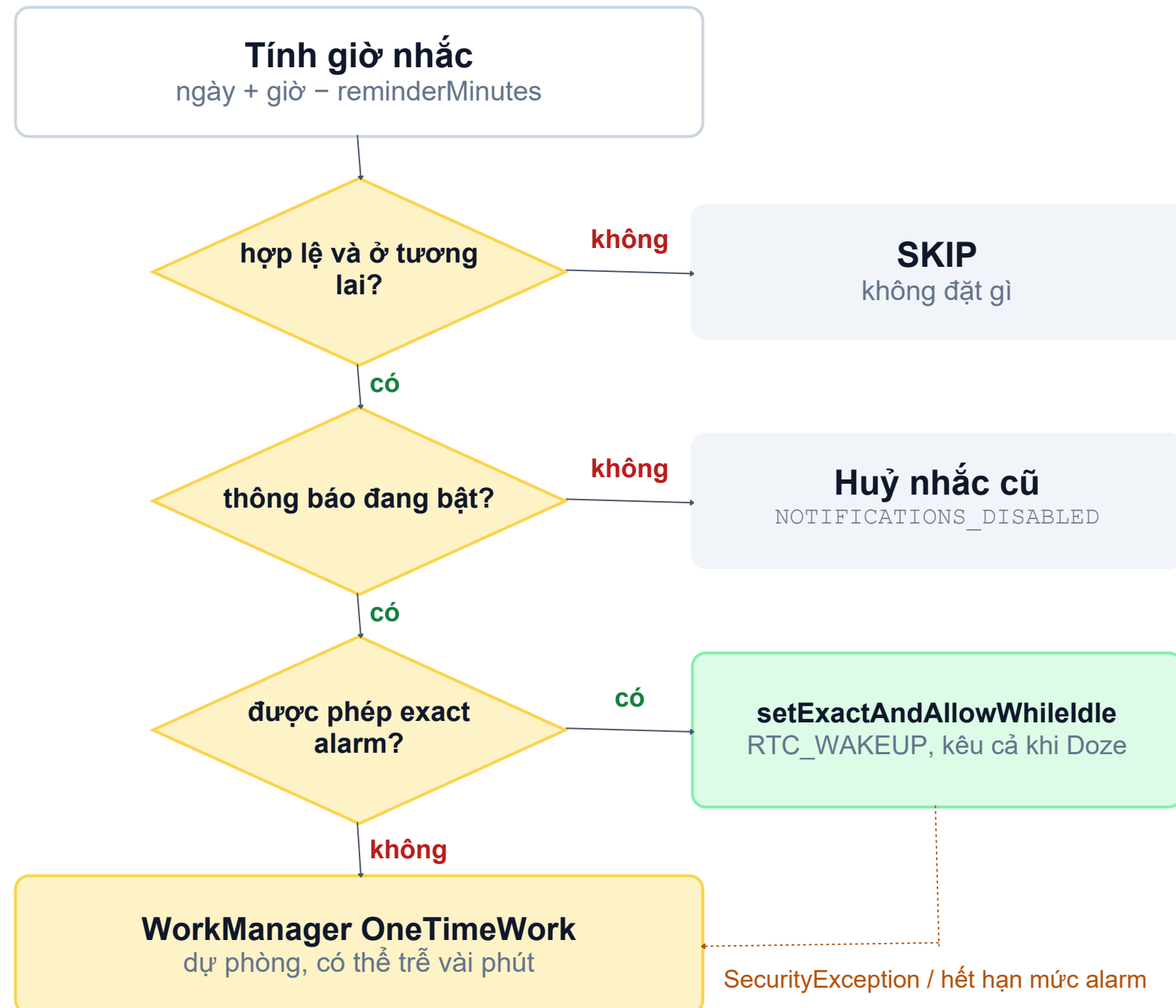
Due date: earliest first

No filters selected

Show 16 tasks

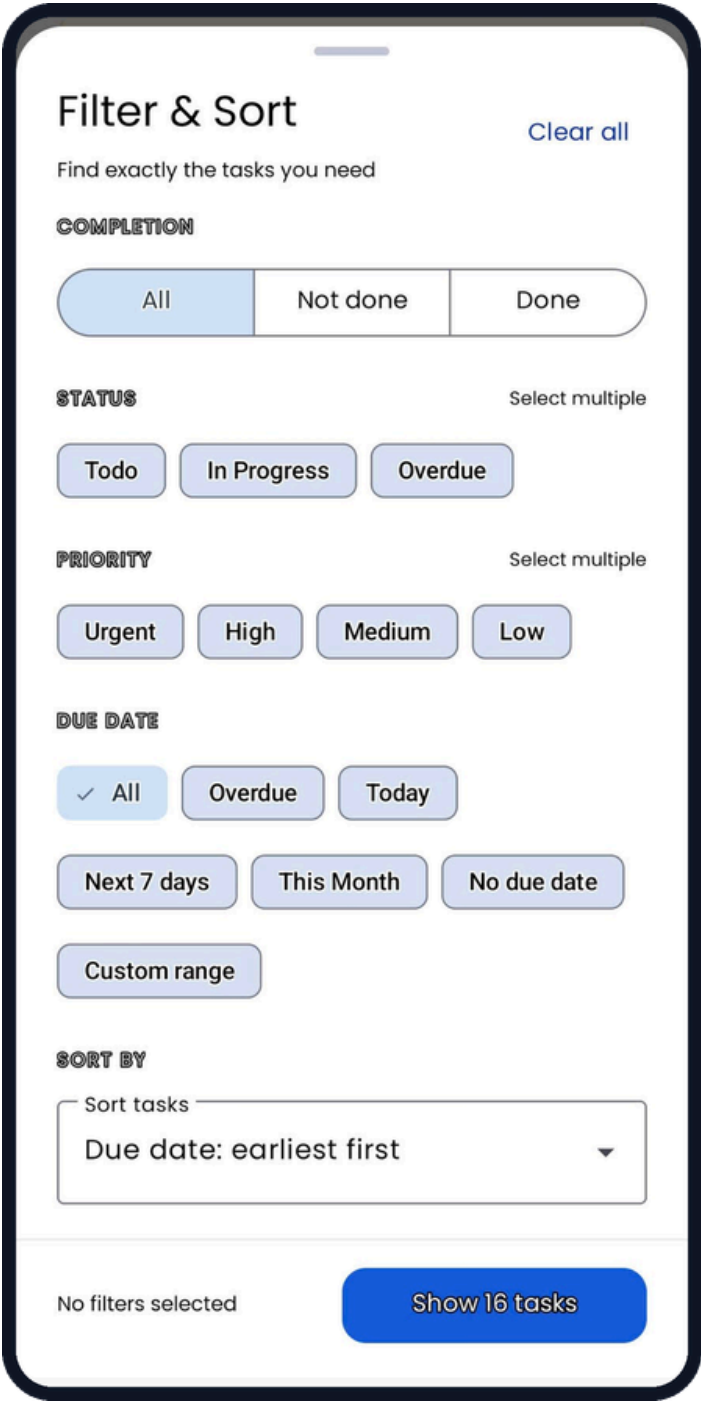
Nhắc nhở: chọn cách hẹn giờ tốt nhất máy cho phép

Từ Android 12, exact alarm có thể bị chặn, nên AlarmScheduler luôn có đường dự phòng bằng WorkManager.



Bộ lọc nhiều tiêu chí và kiểm tra dữ liệu nhập

Trong cùng một nhóm là HOẶC, giữa các nhóm là VÀ. Nhóm để trống nghĩa là không lọc theo nhóm đó.



Hoàn thành

Tất cả

Chưa xong

Đã xong

Trạng thái (để trống = tất cả)

Todo

In Progress

Overdue

Ưu tiên

Urgent

High

Medium

Low

Hạn

Hôm nay

7 ngày tới

Tháng này

Tuỳ chọn

Kết quả của ví dụ trên

task chưa xong, ưu tiên Urgent hoặc High, hạn trong 7 ngày tới.
Task đã xong luôn xếp cuối danh sách.

VÀ

VÀ

VÀ

Chặn tổ hợp vô nghĩa

```
// FilterCriteria.normalized() (rút gọn)
if (completion == DONE) copy(
    statuses = emptySet(),
    dueDateRange =
        if (dueDateRange == OVERDUE) ALL
        else dueDateRange
) else this
```

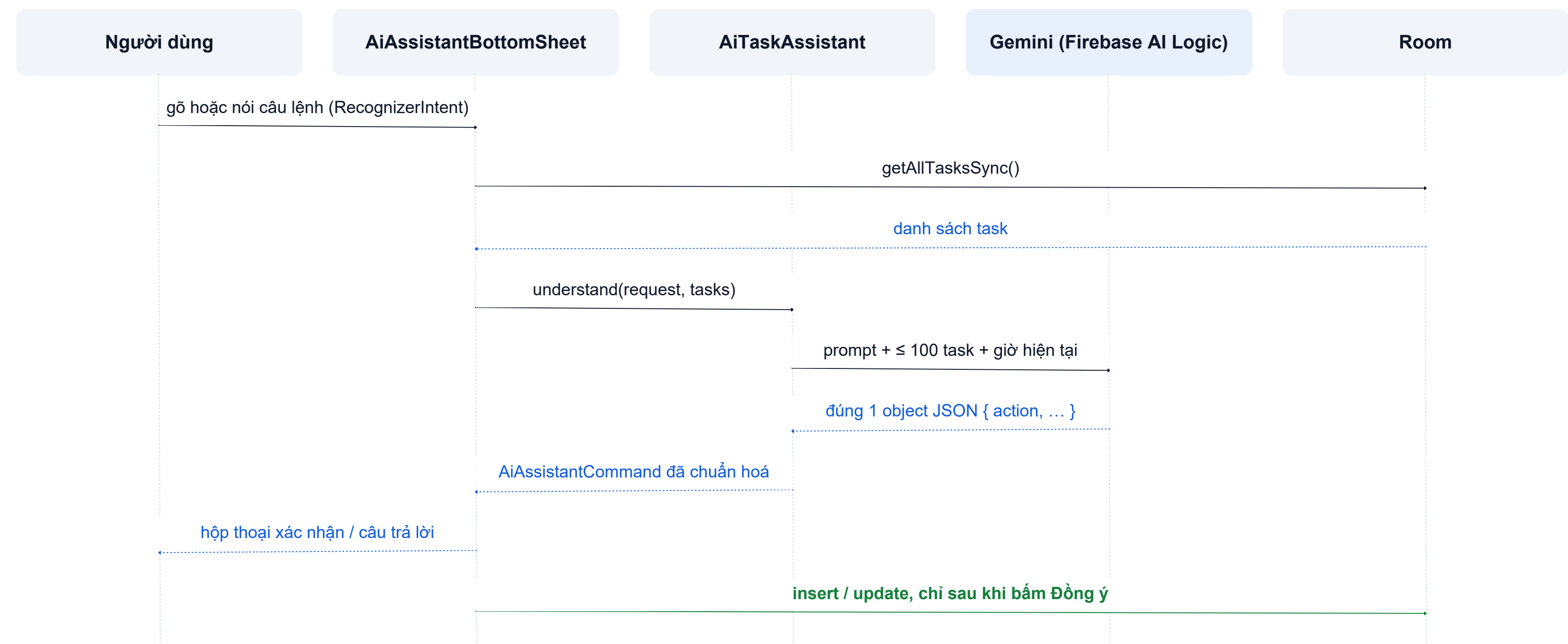
“Đã xong” và “Quá hạn” không thể cùng đúng, nên bộ lọc tự bỏ điều kiện mâu thuẫn thay vì trả về danh sách rỗng khó hiểu.

Kiểm tra dữ liệu nhập (ValidationHelper)

- Tiêu đề bắt buộc, tối đa 200 ký tự; mô tả tối đa 1 000.
- Có ngày hạn; task mới không được đặt ngày đã qua.
- Nếu hạn là hôm nay, giờ phải sau giờ hiện tại.
- 8 unit test cho các luật này.

Trợ lý AI: Gemini hiểu câu lệnh, app tự thực thi

Gemini chỉ trả về một object JSON mô tả ý định. Đọc và ghi dữ liệu hoàn toàn do code của app làm.



AI chỉ đề xuất, người dùng quyết định

Ba nhóm lệnh chính. JSON dưới đây là ví dụ minh họa theo đúng schema app gửi cho Gemini.

CREATE_TASK

“Mai 8 giờ sáng nộp báo cáo, ưu tiên cao”

```
{ "action": "CREATE_TASK",
  "title": "Nộp báo cáo",
  "dueDate": "2026-09-25",
  "dueTime": "08:00",
  "priority": "HIGH" }
```

Hộp thoại xác nhận → insert và đặt nhắc

UPDATE_TASK

“Đánh dấu task Test là xong”

```
{ "action": "UPDATE_TASK",
  "targetTaskId": 2,
  "status": "COMPLETED" }
```

Tìm task theo id trong danh sách đã gửi → xác nhận
→ update

LIST_TASKS

“Còn bao nhiêu task quá hạn?”

```
{ "action": "LIST_TASKS",
  "query": {
    "resultType": "COUNT",
    "datePreset": "OVERDUE" } }
```

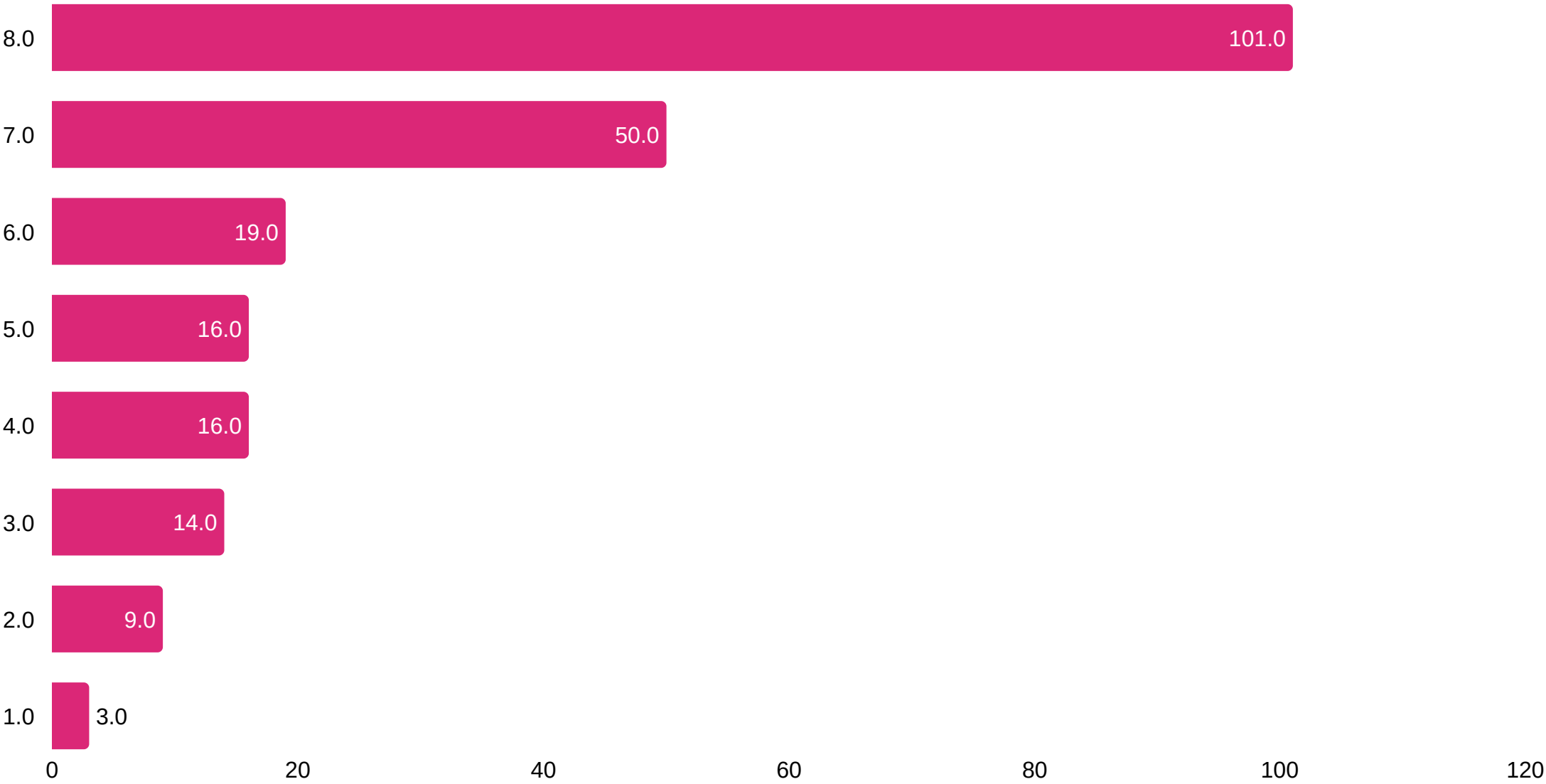
App tự lọc dữ liệu Room bằng matchesQuery() rồi
đếm; con số không do AI đoán

An toàn: không có lệnh xóa · ghi dữ liệu chỉ sau khi người dùng xác nhận · Firebase App Check (Play Integrity, debug token khi dev) chặn gọi API trái phép · google-services.json không đưa lên Git

229 unit test, 34/34 ca kiểm thử thủ công đạt

Logic nghiệp vụ tách khỏi Android nên phần lớn test chạy trên JVM, không cần máy ảo.

Số unit test theo nhóm tính năng



Nguồn: đếm @Test trong 26 file test của repo (tính cả 1 test mẫu).

Kiểm thử hộp đen trên thiết bị

Quản lý công việc	6/6
Thông báo & khôi phục nhắc	4/4
PIN & vân tay	3/3
Backup / restore JSON	5/5
Widget, thống kê, Pomodoro	3/3
Ca biên & chịu lỗi	13/13

Thiết bị

Xiaomi Redmi Note 11 (Android 13), Samsung Galaxy S21 (Android 14), emulator Pixel 6 Pro (API 34).

Nguyên tắc thiết kế và nợ kỹ thuật

Bốn nguyên tắc nhóm giữ xuyên suốt project, và những chỗ nhóm biết là chưa tốt.

4 nguyên tắc xuyên suốt

- 01

Tác dụng phụ gom vào helper dùng chung

AlarmScheduler, NotificationHelper, TaskCompletionHelper được mọi receiver, widget và trợ lý AI gọi lại, không viết lại logic.
- 02

Logic tách khỏi Android

Pomodoro engine, validator, calculator là Kotlin thuần, nên 229 unit test chạy trên JVM.
- 03

Thời gian dùng mốc tuyệt đối

elapsedRealtime() cho bộ đếm, LocalDate cho lịch lập; không cộng trừ mili-giây.
- 04

Kiểm tra trước, ghi sau

Import và restore validate toàn bộ file, rồi ghi trong một transaction.

Nợ kỹ thuật đã biết

VẤN ĐỀ	RỦI RO	HƯỚNG SỬA
PIN 4 số băm bằng SHA-256	chỉ 10 000 tổ hợp, SHA-256 băm rất nhanh nên dò được nếu lộ hash	PBKDF2 / Argon2 (hàm băm chậm)
Chống trùng task lập dựa vào title + chu kỳ	hai chuỗi trùng tên có thể nhận nhầm bản tương lai của nhau	thêm cột seriesId
StreakReminderReceiver không gọi goAsync()	tiến trình có thể bị kill trước khi gửi thông báo	goAsync() như BroadcastReceiver
Logic tick hoàn thành có 2 bản	TaskViewModel và TaskCompletionHelper làm cùng việc; sửa một bên dễ quên bên kia	TaskViewModel gọi TaskCompletionHelper
Chưa có DI framework, chỉ 1 module	truyền dependency thủ công, khó mở rộng	Hilt + tách module data / feature

Hiện tại hash PIN vẫn nằm trong file được Keystore mã hoá, nên rủi ro thực tế thấp.

Kết quả và hướng phát triển

11/11

yêu cầu bắt buộc của đề tài

6

phân hệ mở rộng

229

unit test, chạy trên JVM

358

commit từ 5 thành viên

Còn hạn chế

- Chưa đồng bộ đám mây giữa nhiều thiết bị.
- Chưa có làm việc nhóm, chia sẻ hay giao task.
- Trợ lý AI cần mạng và cấu hình Firebase riêng cho từng máy dev.
- Chưa đính kèm ảnh, file hay ghi âm vào task.

Hướng phát triển

- Đồng bộ tùy chọn qua Google Drive hoặc Firestore, có xử lý xung đột.
- Đính kèm ảnh, ghi âm; gắn tag để lọc.
- Ứng dụng đồng hành trên Wear OS.

Cách nhóm làm việc: chia việc trên Jira; mỗi tính năng một branch riêng, merge vào main qua Pull Request, trưởng nhóm review và xử lý conflict.

Điều nhóm rút ra

- Merge vào main thường xuyên: branch để lâu gây conflict lớn ở AndroidManifest, MainActivity, libs.versions.toml.
- Có bản thiết kế UI chung (Stitch) từ đầu nên 5 người làm giao diện vẫn đồng bộ.
- Migration phải chạy lại được: kiểm tra cột trước khi ALTER TABLE.

Demo & Hỏi đáp

Cảm ơn thầy và các bạn đã lắng nghe.

Thứ tự demo

- 1 Mở app, mở khoá bằng vân tay
- 2 Tạo task có nhắc sau 1 phút
- 3 Nhận thông báo, bấm Hoàn thành
- 4 Tick một task ngay trên widget
- 5 Chạy Pomodoro rồi thoát ra ngoài
- 6 Hỏi trợ lý AI: “hôm nay còn mấy task?”

● **Trần Văn Thái**
Trưởng nhóm · Kiến trúc, bảo mật, widget

● **Trần Văn Ngọc Thắng**
Feature Developer · Task, lịch, vân tay

● **Nguyễn Lê Huy Tâm**
Feature Developer · Task lặp, import, streak

● **Huỳnh Đình Chấn**
Feature Developer · Pomodoro, độ bền dữ liệu

● **Nguyễn Ngọc Gia Bảo**
QA & Feature Developer · Nhắc nhở, lọc, AI

github.com/ThaiDevv/Task-Management-App